অধ্যায় ২৭

২৭

Real-Time JavaScript Applications

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- Short Polling, Long Polling, Server-Sent Events (SSE) ও WebSocket-এর পার্থক্য ও উপযুক্ত ব্যবহার বিচার করতে।
- Node.js-এ SSE Stream ও WebSocket Server তৈরি করতে, এবং Browser ও Node.js-এর WebSocket Client দিয়ে সংযোগ করতে।
- WebSocket-এর Lifecycle (Handshake, `open`, `message`, `close`), Close Code ও Heartbeat বুঝতে।
- Socket.IO-র Event, Room ও Acknowledgement-এর ধারণা বুঝতে, এবং কেন তা Raw WebSocket-এর সঙ্গে সরাসরি মেলে না তা ব্যাখ্যা করতে।
- একটি স্থির Event Envelope (eventId, type, version, sequence) নকশা ও যাচাই করতে।
- Command ও Event আলাদা করতে, এবং Transactional Outbox দিয়ে "Database-এ লেখা হলো কিন্তু Event গেল না" সমস্যা এড়াতে।
- At-most-once, At-least-once ও Exactly-once Effect বুঝতে; Duplicate, পুরোনো ও হারানো Event সামলাতে।
- Exponential Backoff ও Jitter দিয়ে পুনঃসংযোগ, এবং Bounded Buffer দিয়ে Backpressure সামলাতে।
- একাধিক Server Instance-এ Pub/Sub দিয়ে Event ছড়ানোর নকশা ও তার সীমা বুঝতে।
- একটি সম্পূর্ণ পরীক্ষিত **TravelLive** Real-Time Booking Dashboard তৈরি করতে।

পূর্বশর্ত

- অধ্যায় ১৩ ও ১৪: Promise, `async` / `await`, Event Loop।
- অধ্যায় ১৭ ও ২০: Browser-এর DOM Event, Input Validation, Authentication ও Authorization।
- অধ্যায় ২২: `node:test`, Integration Test।
- অধ্যায় ২৫: Node.js HTTP Server, Middleware, JSON Error চুক্তি, Idempotency।
- অধ্যায় ২৬: Transaction ও `node:sqlite`।

এই অধ্যায় কেন গুরুত্বপূর্ণ

একটি ভ্রমণ সংস্থার অফিস। Agent নাদিয়া একটি Booking নিশ্চিত করলেন। একই মুহূর্তে গ্রাহক রাফি তাঁর Phone-এ Booking-এর পাতা খুলে বসে আছেন, আর পাশের Desk-এ আরেকজন Agent একই Booking বাতিলের কথা ভাবছেন। রাফির Screen-এ যদি পাঁচ মিনিট পরেও "অপেক্ষমাণ" লেখা থাকে, তিনি ফোন করবেন; দ্বিতীয় Agent যদি পুরোনো অবস্থা দেখে সিদ্ধান্ত নেন, ভুল হবে।

Page বারবার Refresh করা বা প্রতি কয়েক সেকেন্ডে Server-কে জিজ্ঞেস করা সমাধান নয়—তাতে হাজারো অপ্রয়োজনীয় অনুরোধ যায়, আর তবুও দেরি হয়। **Real-Time** Application-এ Server নিজেই পরিবর্তনের খবর Client-এ পৌঁছে দেয়।

কিন্তু শুধু একটি Socket খুলে Message পাঠালেই কাজ শেষ নয়। Network কেটে যায় এবং ফিরে আসে; একই Event দুবার পৌঁছায়; কখনো একটি হারিয়ে যায়; ধীর Client-এর জন্য Server-এর Memory ভরে ওঠে; আর সংযুক্ত থাকা মানেই সবকিছু দেখার অনুমতি থাকা নয়। এই অধ্যায় এই প্রতিটি সমস্যা আসল Code ও আসল Network সংযোগ দিয়ে দেখায়, এবং শেষে একটি Browser Dashboard তৈরি করে যা এই সব পরিস্থিতিতে সঠিক থাকে।

২৭.১ আগে যা শিখেছেন, এখন যা নতুন

আগে শেখা	এই অধ্যায়ে নতুন
HTTP Request ও Response (অধ্যায় ২৫)	দীর্ঘস্থায়ী সংযোগ: SSE Stream ও WebSocket
Token দিয়ে Authentication (অধ্যায় ২০)	সংযোগের প্রথম Message-এ Auth; প্রতিটি Subscribe ও Dispatch-এ অনুমতি
Idempotency-Key (অধ্যায় ২৫, ২৬)	Event-এর eventId দিয়ে Duplicate বাদ
Transaction (অধ্যায় ২৬)	Transactional Outbox
Promise ও Timer (অধ্যায় ১৩, ১৪)	Heartbeat, Backoff, Jitter
Integration Test (অধ্যায় ২২)	আসল WebSocket সংযোগে Test, আসল Browser-এ End-to-End

> **নোট:** WebSocket Server-এর জন্য এই অধ্যায় জনপ্রিয় `ws` Package ব্যবহার করে (`npm install ws`)। QA-যন্ত্রে নতুন কোনো Package Install করা হয়নি; আগে থেকে থাকা `ws` 7.5.10 দিয়ে সব Code চালানো হয়েছে, এবং Code শুধু সেই API ব্যবহার করে যা `ws` -এর ৭ ও ৮ দুই সংস্করণেই আছে। Client হিসেবে Node.js 22-এর অন্তর্নির্মিত `WebSocket` ও আসল Chrome Browser ব্যবহৃত হয়েছে। Socket.IO ও Redis QA-যন্ত্রে ছিল না: Socket.IO-র Code শুধু নকশা হিসেবে দেখানো হয়েছে (চালানো হয়নি), আর Redis Pub/Sub-এর আচরণ একটি ছোট in-memory অনুকরণ দিয়ে দেখানো হয়েছে—যেখানে যা, স্পষ্ট লেখা আছে।

২৭.২ Real-Time-এর চারটি পথ

Server থেকে Client-এ নতুন খবর পৌঁছানোর চারটি প্রচলিত কৌশল:

কৌশল	কীভাবে	কখন উপযুক্ত	সীমা
Short Polling	Client নির্দিষ্ট বিরতিতে বারবার HTTP অনুরোধ	কদাচিৎ পরিবর্তন, খুব সাধারণ Client	অপ্রয়োজনীয় অনুরোধ, বিরতির অর্ধেক গড় দেরি
Long Polling	Server উত্তর আটকে রাখে, খবর এলে বা সময় শেষে উত্তর	পুরোনো পরিবেশে বিকল্প	প্রতিটি খবরের পর নতুন অনুরোধ
SSE	একটি HTTP Response যা শেষ হয় না; Server Event লিখতে থাকে	Notification, Status Feed, Dashboard	একমুখী (Server থেকে Client), শুধু Text
WebSocket	HTTP থেকে Upgrade করা দ্বিমুখী স্থায়ী সংযোগ	Chat, যৌথ সম্পাদনা, Live Dashboard	সংযোগের জীবনচক্র ও Scaling নিজে সামলাতে হয়

Polling-এর খরচ সংখ্যায় দেখা যাক। ১০০০ জন Operator এক ঘণ্টা Dashboard খোলা রাখেন, আর সেই ঘণ্টায় আসল পরিবর্তন মাত্র ৪০টি:

polling-cost.mjs · JavaScript চলবে: Node.js

```
// ১০০০ জন Operator, ১ ঘণ্টা, আর ঘণ্টায় মাত্র ৪০টি আসল পরিবর্তন।
const clients = 1000;
const seconds = 60 * 60;
const changes = 40;

function polling(intervalSeconds) {
  const requests = clients * Math.floor(seconds / intervalSeconds);
  const useful = clients * changes;
  return { requests, wasted: requests - useful,
    averageDelay: intervalSeconds / 2 };
}

console.log('Polling ৫ সেকেন্ড:', polling(5));
console.log('Polling ৬০ সেকেন্ড:', polling(60));
// Push (SSE/WebSocket): প্রতিটি পরিবর্তন একবার প্রতিটি Client-এ।
console.log('Push:', { messages: clients * changes, averageDelay: 0 });
```

Output

```
Polling ৫ সেকেন্ড: { requests: 720000, wasted: 680000, averageDelay: 2.5 }
Polling ৬০ সেকেন্ড: { requests: 60000, wasted: 20000, averageDelay: 30 }
Push: { messages: 40000, averageDelay: 0 }
```

প্রতি ৫ সেকেন্ডের Polling-এ ৭ লাখ ২০ হাজার অনুরোধের ৬ লাখ ৮০ হাজারই অর্থহীন, তবু গড়ে আড়াই সেকেন্ড দেরি। বিরতি বাড়ালে অনুরোধ কমে, কিন্তু দেরি আধা মিনিট। Push-এ (SSE বা WebSocket) ঠিক যতগুলো পরিবর্তন, ততগুলো Message, এবং দেরি প্রায় শূন্য। তবু Polling-কে তুচ্ছ ভাববেন না: দিনে কয়েকবার বদলায় এমন তথ্যের জন্য, বা যেখানে Proxy দীর্ঘ সংযোগ কেটে দেয়, Polling সবচেয়ে সরল ও নির্ভরযোগ্য।

সিদ্ধান্তের নিয়ম: শুধু Server থেকে খবর আসবে? SSE দিয়ে শুরু করুন—এটি সাধারণ HTTP, Browser নিজেই পুনঃসংযোগ করে। দুই দিকেই ঘন ঘন Message? WebSocket। Room, স্বয়ংক্রিয় পুনঃসংযোগ ও Fallback একসঙ্গে চাই এবং দুই পাশেই একই Library ব্যবহার করতে পারবেন? Socket.IO।

২৭.৩ Server-Sent Events

SSE হলো একটি সাধারণ HTTP Response, যার `Content-Type` হলো `text/event-stream` এবং যা শেষ হয় না। প্রতিটি Event কয়েকটি `field: value` Line, আর একটি ফাঁকা Line (অর্থাৎ `\n\n`) Event শেষ করে। প্রধান Field চারটি: `data` (তথ্য), `event` (Event-এর নাম), `id` (Event-এর পরিচয়) ও `retry` (পুনঃসংযোগের আগে কত Millisecond অপেক্ষা)। `:` দিয়ে শুরু Line মন্তব্য—সংযোগ জীবিত রাখতে ব্যবহৃত হয়।

sse-basic.mjs · JavaScript চলবে: Node.js

```
import http from 'node:http';
```

```
// Server: সাধারণ HTTP Response, যা শেষ হয় না; প্রতিটি Event "\n\n" দিয়ে আলাদা।
const server = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/event-stream',
    'Cache-Control': 'no-cache' });
  const statuses = ['PENDING', 'CONFIRMED', 'TICKETED'];
  statuses.forEach((status, i) => setTimeout(() => {
    res.write(`id: ${i + 1}\nevent: status\ndata: {"status":"${status}"}\n\n`);
    if (i === statuses.length - 1) res.end();
  }, 20 * i));
});
await new Promise((resolve) => server.listen(0, '127.0.0.1', resolve));

// Client: Browser-এ EventSource; এখানে fetch দিয়ে Stream পড়ে নিজেই ভাগ করা।
const res = await fetch(`http://127.0.0.1:${server.address().port}/`);
console.log('Content-Type:', res.headers.get('content-type'));
let buffer = '';
for await (const chunk of res.body.pipeThrough(new TextDecoderStream())) {
  buffer += chunk;
  const frames = buffer.split('\n\n');
  buffer = frames.pop();
  for (const frame of frames) {
    const fields = Object.fromEntries(frame.split('\n')
      .map((line) => [line.slice(0, line.indexOf(':')),
        line.slice(line.indexOf(':') + 2)]));
    console.log(`#${fields.id} ${fields.event}:`, JSON.parse(fields.data));
  }
}
server.close();
```

Output

```
Content-Type: text/event-stream
#1 status: { status: 'PENDING' }
#2 status: { status: 'CONFIRMED' }
#3 status: { status: 'TICKETED' }
```

Browser-এ এই ভাগ করার কাজ `EventSource` নিজেই করে: `new EventSource('/api/bookings/bk-1/stream')` এবং `source.addEventListener('status', handler)`। সংযোগ কাটলে Browser নিজেই আবার সংযোগ করে, এবং শেষ পাওয়া `id` একটি `Last-Event-ID` Header-এ পাঠায়—Server সেটি দেখে বাদ পড়া Event আবার পাঠাতে পারে। TravelLive Project ঠিক এটিই করে (অনুচ্ছেদ ২৭.১২)।

দুটি সীমা মনে রাখুন। প্রথমত, `EventSource` কোনো Custom Header (যেমন `Authorization`) পাঠাতে পারে না; Browser-এ Authentication সাধারণত Cookie দিয়ে হয়, যার জন্য একই Origin বা সঠিক CORS ও `withCredentials` লাগে। Token URL-এ বসাবেন না—URL Server Log ও Browser History-তে থেকে যায়। দ্বিতীয়ত, HTTP/1.1-এ একটি Browser একই Domain-এ সীমিত সংখ্যক (সাধারণত ছয়টি) সংযোগ খোলে; অনেক Tab-এ SSE খুললে এই সীমায় পৌঁছানো যায়। HTTP/2-তে এ সমস্যা প্রায় নেই।

২৭.৪ WebSocket: Handshake ও Lifecycle

WebSocket শুরু হয় একটি সাধারণ HTTP অনুরোধ দিয়ে, যাতে `Connection: Upgrade` ও `Upgrade: websocket` Header থাকে। Server রাজি হলে `101 Switching Protocols` দিয়ে উত্তর দেয়, এবং সেই TCP সংযোগ এরপর আর HTTP থাকে না—দুই পক্ষই যেকোনো সময় Frame পাঠাতে পারে। Browser সব সময় এই অনুরোধে `Origin` Header পাঠায়; Server-এর উচিত তা যাচাই করা।

Client-এর দিকে চারটি Lifecycle Event: `open`, `message`, `error`, `close`। `readyState`-এর মান 0 (CONNECTING), 1 (OPEN), 2 (CLOSING), 3 (CLOSED)। নিচের উদাহরণে Server `ws` Package দিয়ে, আর Client Node.js 22-এর অন্তর্নির্মিত `WebSocket`—যার API Browser-এর `WebSocket`-এর হুবহু এক:

ws-lifecycle.mjs · JavaScript — চলবে: Node.js

```
import http from 'node:http';
import WebSocketLib from 'ws';

// Server: 'ws' Package (npm install ws)। Client: Node.js 22-এর অন্তর্নির্মিত
// WebSocket, যা Browser-এর মতোই addEventListener ব্যবহার করে।
const server = http.createServer();
const wss = new WebSocketLib.Server({ server });
wss.on('connection', (socket) => {
  socket.send(JSON.stringify({ type: 'welcome' }));
  socket.on('message', (data) => {
    const msg = JSON.parse(String(data));
    socket.send(JSON.stringify({ type: 'echo', text: msg.text }));
    socket.close(1000, 'done');
  });
});
await new Promise((resolve) => server.listen(0, '127.0.0.1', resolve));

const ws = new WebSocket(`ws://127.0.0.1:${server.address().port}`);
console.log('শুরুতে readyState =', ws.readyState);
ws.addEventListener('open', () => console.log('open, readyState =',
  ws.readyState));
ws.addEventListener('message', (event) => {
  const msg = JSON.parse(event.data);
  console.log('message:', msg);
  if (msg.type === 'welcome') ws.send(JSON.stringify({ text: 'হ্যালো' }));
});
ws.addEventListener('close', (event) => {
  console.log('close:', event.code, event.reason, 'clean =', event.wasClean);
  server.close();
});
```

Output

```
শুরুতে readyState = 0
open, readyState = 1
message: { type: 'welcome' }
message: { type: 'echo', text: 'হ্যালো' }
close: 1000 done clean = true
```

লক্ষ করুন, `new WebSocket(...)`-এর ঠিক পরে `readyState` 0: সংযোগ তখনো তৈরি হচ্ছে। এই অবস্থায় `send()` করলে Browser Error দেয়; তাই প্রথম Message সব সময় `open`-এর পরে।

সংযোগ বন্ধের সময় একটি **Close Code** ও কারণ পাঠানো হয়। কিছু Code Protocol-এর নির্ধারিত, আর 4000 থেকে 4999 পর্যন্ত Application নিজের অর্থে ব্যবহার করতে পারে:

Code	অর্থ	কে পাঠায়
1000	স্বাভাবিক বন্ধ	যেকোনো পক্ষ
1001	চলে যাচ্ছে (Page বন্ধ, Server Restart)	যেকোনো পক্ষ
1006	অস্বাভাবিক বন্ধ, Close Frame আসেনি (Network কাটা)	কেউ পাঠায় না; Client নিজে জানায়
1009	Message খুব বড়	গ্রহণকারী
1011	Server-এর অপ্রত্যাশিত সমস্যা	Server
4401, 4429	TravelLive-এ: Auth ব্যর্থ, Rate Limit	TravelLive Server

Production-এ সব সময় `wss://` (TLS-এর ওপর WebSocket) ব্যবহার করুন; `ws://` শুধু নিজের Computer-এ। প্রতিটি আসা Message-এর আকার সীমিত রাখুন, JSON-এর ধরন যাচাই করুন এবং প্রতি সংযোগে Message-এর হার সীমিত করুন। শুধু Client-এ যাচাই নিরাপত্তা দেয় না—যে কেউ Browser ছাড়াই একটি WebSocket Client লিখে যা খুশি পাঠাতে পারে।

২৭.৫ Socket.IO: Event, Room ও Acknowledgement

Socket.IO একটি Library, যা WebSocket-এর ওপর (প্রয়োজনে HTTP Long Polling-এ নেমে) নিজস্ব Protocol চালায়। এটি দেয়: নামযুক্ত Event (`socket.emit('booking:subscribe', data)`), **Room** (একদল Socket-কে একসঙ্গে Message), **Acknowledgement** (প্রাপক একটি Callback দিয়ে উত্তর দেয়), স্বয়ংক্রিয় পুনঃসংযোগ, এবং একাধিক Server-এ Room-এর Message পৌঁছাতে Adapter (যেমন Redis Adapter)।

গুরুত্বপূর্ণ: Socket.IO-র নিজস্ব Handshake, Frame-এর গঠন ও Event-এর Format আছে। তাই Socket.IO Client কোনো সাধারণ WebSocket Server-এ সংযোগ করতে পারে না, এবং Browser-এর সাধারণ `WebSocket` দিয়ে Socket.IO Server-এ কথা বলা যায় না—দুই পাশেই Socket.IO লাগে।

নিচের Code শুধু নকশা দেখানোর জন্য; QA-যন্ত্রে `socket.io` ছিল না বলে এটি চালানো হয়নি, শুধু Node.js-এর Syntax-যাচাই পার হয়েছে:

socketio-design.mjs · JavaScript — চলবে: Node.js

```
// শুধু নকশা দেখানোর জন্য; চালানো হয়নি। এই বইয়ের QA-যন্ত্রে socket.io নেই,
// এবং কোনো Package Install করা হয়নি। চালাতে: npm install socket.io
// (session-auth.mjs আপনার নিজের Session ও অনুমতি-যাচাইয়ের Module।)
import { createServer } from 'node:http';
import { Server } from 'socket.io';
import { authenticateSession, canViewBooking } from './session-auth.mjs';

const httpServer = createServer();
const io = new Server(httpServer, {
  cors: { origin: ['https://app.example.com'] },
  maxHttpBufferSize: 4 * 1024,
});

// Middleware: Connection-এর আগেই Session যাচাই।
io.use(async (socket, next) => {
  try {
    socket.data.user = await authenticateSession(socket.request);
    next();
  } catch {
    next(new Error('UNAUTHORIZED'));
  }
});

io.on('connection', (socket) => {
  // Acknowledgement: Client-এর কাছে ফলাফল ফেরত (ack Callback)।
  socket.on('booking:subscribe', async ({ bookingId } = {}, ack) => {
    if (typeof bookingId !== 'string') return ack?.({ ok: false });
    if (!await canViewBooking(socket.data.user.id, bookingId)) {
      return ack?.({ ok: false, code: 'FORBIDDEN' });
    }
    await socket.join(`booking:${bookingId}`);
    return ack?.({ ok: true });
  });
});

// Room-এ পাঠানো: ওই Room-এর সব Socket পায়, যে Instance-এই থাকুক
// (একাধিক Instance-এ Redis Adapter লাগবে)।
export function publish(event) {
  io.to(`booking:${event.bookingId}`).emit('booking:event', event);
}

httpServer.listen(3000);
```

তিনটি বিষয় খেয়াল করুন। এক, `io.use` Middleware সংযোগের আগেই Session যাচাই করে। দুই, Room-এ যোগ দেওয়ার আগে Server যাচাই করে ব্যবহারকারী Booking-টি দেখতে পারেন কি না—**Room-এর নাম গোপন রাখা অনুমতি নয়**; কেউ অনুমান করে `booking:bk-22` চাইতেই পারে। তিন, Room-এর সদস্যপদ একটি সংযোগের; পুনঃসংযোগ হলে Client-কে আবার Subscribe করতে হয়, এবং Server-কে আবার অনুমতি যাচাই করতে হয়। TravelLive এই একই নকশা `ws` দিয়ে নিজে তৈরি করে, যাতে প্রতিটি অংশ আপনি দেখতে ও পরীক্ষা করতে পারেন।

২৭.৬ Event Contract ও Versioning

Client ও Server যেন একই ভাষায় কথা বলে, তাই প্রতিটি Event-এর একটি স্থির খাম (**Envelope**) থাকা উচিত:

events.mjs · JavaScript — চলবে: Node.js

```
// Event Envelope: প্রতিটি Event-এর একই কাঠামো, যাতে Client নিশ্চিন্তে পড়তে পারে।
export const EVENT_VERSION = 1;

export function createEvent({ id, type, bookingId, sequence, payload, now }) {
  return Object.freeze({
    eventId: id,
    type,
    version: EVENT_VERSION,
    bookingId,
    sequence,
    occurredAt: now.toISOString(),
    payload: Object.freeze({ ...payload }),
  });
}

// Network থেকে আসা Event বিশ্বাস করবেন না: আকার ও ধরন যাচাই করুন।
export function isValidEvent(value) {
  return value !== null && typeof value === 'object'
    && typeof value.eventId === 'string' && value.eventId.length > 0
    && typeof value.type === 'string'
    && value.version === EVENT_VERSION
    && typeof value.bookingId === 'string'
    && Number.isInteger(value.sequence) && value.sequence > 0
    && typeof value.payload === 'object' && value.payload !== null;
}
```

প্রতিটি Field-এর আলাদা কাজ: `eventId` Duplicate চিনতে; `type` কোন ঘটনা (অতীতকালে নাম: `booking.status.changed`); `version` Payload-এর গঠন বদলালে পুরোনো ও নতুন Client আলাদা করতে; `bookingId` কোন Aggregate-এর Event; `sequence` সেই Booking-এর মধ্যে ক্রম ও ফাঁক ধরতে; `occurredAt` কখন ঘটেছে; এবং `payload` শুধু প্রয়োজনীয় তথ্য।

Network থেকে আসা Event-কে বিশ্বাস করার আগে যাচাই করুন:

envelope.mjs · JavaScript চলবে: Node.js

```
import { createEvent, isValidEvent }
  from '../project/travel-live/src/events.mjs';

const event = createEvent({ id: 'evt-102', type: 'booking.status.changed',
  bookingId: 'bk-22', sequence: 8, payload: { status: 'CONFIRMED' },
  now: new Date('2026-10-11T00:00:00Z') });
console.log(JSON.stringify(event, null, 2));

// Network থেকে আসা কয়েকটি সন্দেহজনক Message।
const incoming = [
  JSON.stringify(event),
  '{"type":"booking.status.changed","sequence":"8"}',
  JSON.stringify({ ...event, version: 2 }),
  JSON.stringify({ ...event, sequence: 0 }),
];
for (const text of incoming) {
  const value = JSON.parse(text);
  console.log(isValidEvent(value) ? 'গ্রহণ' : 'বাতিল', value.sequence,
    value.version);
}
```

Output

```
{
  "eventId": "evt-102",
  "type": "booking.status.changed",
  "version": 1,
  "bookingId": "bk-22",
  "sequence": 8,
  "occurredAt": "2026-10-11T00:00:00.000Z",
  "payload": {
    "status": "CONFIRMED"
  }
}
গ্রহণ 8 1
বাতিল 8 undefined
বাতিল 8 2
বাতিল 0 1
```

দ্বিতীয় Message-এ `sequence` একটি String—JSON-এ খুব সাধারণ ভুল। তৃতীয়টি `version` 2: এই Client সেটি বোঝে না, তাই অনুমান না করে বাতিল করে (বাস্তবে Log করে, এবং প্রয়োজনে পূর্ণ Snapshot চায়)। **Versioning-এর নিয়ম:** নতুন ঐচ্ছিক Field যোগ করা নিরাপদ, তাই version একই থাকতে পারে; কোনো Field-এর নাম বা অর্থ বদলালে বা সরালে version বাড়ান, এবং কিছু সময় দুই সংস্করণই পাঠান যতক্ষণ না সব Client হালনাগাদ হয়।

Payload-এ Password, Access Token, পূর্ণ ফোন নম্বর, কার্ডের তথ্য বা অপ্রয়োজনীয় আর্থিক বিবরণ পাঠাবেন না। Event অনেক জায়গায় যায়—Browser-এর DevTools, Log, Analytics—এবং যা একবার গেছে, তা ফেরানো যায় না।

২৭.৭ Event-Driven Architecture ও Transactional Outbox

দুটি শব্দ আলাদা রাখুন। **Command** একটি অনুরোধ: "Booking নিশ্চিত করুন" (`ConfirmBooking`)—এটি প্রত্যাখ্যাত হতে পারে। **Event** একটি ঘটে যাওয়া সত্য: "Booking নিশ্চিত হয়েছে" (`booking.confirmed`)—এটি বদলানো যায় না। Command আসে, যাচাই ও অনুমতি পরীক্ষা হয়, অবস্থা বদলায়, তারপর Event প্রকাশিত হয়। একই Event থেকে Notification, Analytics ও Audit—প্রত্যেকে নিজের মতো কাজ করে, Booking Service তাদের চেনেও না।

Folder-এর কাঠামো

```
Client Command -> API (Auth) -> Booking Service -> Database Transaction
                                                    |  Booking সারি
                                                    |  Outbox সারি
                                                    v
                                          Outbox Relay (আলাদা Worker)
                                                    |
                                                    v
                                       Broker বা WebSocket Gateway
                                                    |
                                                    v
                                          অনুমতিপ্রাপ্ত Client-গুলো
```

এখানে একটি সূক্ষ্ম বিপদ আছে: **Dual Write**। Service যদি প্রথমে Database-এ লেখে, তারপর Broker-এ Event পাঠায়, তাহলে মাঝখানে Process বন্ধ হলে Booking নিশ্চিত হলো কিন্তু কেউ জানল না। উল্টো ক্রমে করলে Event গেল কিন্তু Booking হয়তো Rollback হলো। **Transactional Outbox** এর সমাধান: Business পরিবর্তন ও একটি `outbox` সারি একই Transaction-এ লেখা হয়; একটি আলাদা Relay পরে Outbox থেকে পড়ে প্রকাশ করে।

outbox.mjs · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Node.js

```
import { DatabaseSync } from 'node:sqlite';

const db = new DatabaseSync(':memory:');
db.exec(`CREATE TABLE bookings (id TEXT PRIMARY KEY, status TEXT NOT NULL);
  CREATE TABLE outbox (id INTEGER PRIMARY KEY, type TEXT, payload TEXT,
    published INTEGER NOT NULL DEFAULT 0);
  INSERT INTO bookings VALUES ('bk-1', 'PENDING');`);

// Business পরিবর্তন ও Outbox সারি একই Transaction-এ: দুটোই, নয়তো কোনোটিই নয়।
function confirm(id) {
  db.exec('BEGIN');
  try {
    db.prepare("UPDATE bookings SET status = 'CONFIRMED' WHERE id = ?").run(id);
    db.prepare('INSERT INTO outbox (type, payload) VALUES (?, ?)')
      .run('booking.confirmed', JSON.stringify({ bookingId: id }));
    db.exec('COMMIT');
  } catch (error) {
    db.exec('ROLLBACK');
    throw error;
```

```
  }
}

// ...
```

Relay ও একটি Idempotent Consumer:

outbox.mjs · JavaScript (অংশ ২ / ২ (Part 2 of 2)) চলবে: Node.js

```
// ...

// Relay: আলাদা Worker; Publish সফল হলে তবেই published = 1।
function relay(publish) {
  const rows = db.prepare('SELECT * FROM outbox WHERE published = 0').all();
  for (const row of rows) {
    publish(row);
    db.prepare('UPDATE outbox SET published = 1 WHERE id = ?').run(row.id);
  }
}

const seen = new Set();
const consumer = (row) => {
  if (seen.has(row.id)) return console.log('  duplicate বাদ:', row.id);
  seen.add(row.id);
  console.log('  Email পাঠানো হলো:', row.type, row.payload);
};

confirm('bk-1');
console.log('১ম Relay: Publish-এর পরে, published লেখার আগেই Crash');
try {
  relay((row) => { consumer(row); throw new Error('Crash'); });
} catch (error) {
  console.log('  ', error.message);
}
console.log('২য় Relay (Restart): একই সারি আবার, Consumer Idempotent');
relay(consumer);
console.log('বাকি:', db.prepare(
  'SELECT COUNT(*) AS n FROM outbox WHERE published = 0').get().n);
```

Output

```
১ম Relay: Publish-এর পরে, published লেখার আগেই Crash
  Email পাঠানো হলো: booking.confirmed {"bookingId":"bk-1"}
   Crash
২য় Relay (Restart): একই সারি আবার, Consumer Idempotent
  duplicate বাদ: 1
বাকি: 0
```

এই আউটপুট Outbox-এর আসল চরিত্র দেখায়। Relay Event প্রকাশ করার পরে কিন্তু `published = 1` লেখার আগে বন্ধ হলে, Restart-এর পর একই সারি আবার প্রকাশিত হয়—অর্থাৎ Outbox Event **হারায় না, কিন্তু Duplicate দিতে পারে** (At-least-once)। তাই Consumer-কে Idempotent হতে হয়: এখানে সে সারির `id` মনে রেখে দ্বিতীয় Email পাঠায় না। `node:sqlite` চালালে Node.js একটি "ExperimentalWarning" দেখায়; এটি স্বাভাবিক।

২৭.৮ Delivery Semantics, Ordering ও Duplicate

Message পৌঁছানোর তিনটি নিশ্চয়তা:

- **At-most-once:** একবার পাঠানো, Retry নেই। Duplicate হয় না, কিন্তু হারাতে পারে। (Live Cursor-এর অবস্থান, যেখানে পরেরটি আগেরটিকে অর্থহীন করে।)
- **At-least-once:** নিশ্চিত না হওয়া পর্যন্ত Retry। হারায় না, কিন্তু Duplicate আসতে পারে—তাই Consumer Idempotent। (Booking Event, Payment Notification।)
- **Exactly-once Effect:** Network-এর ওপর সত্যিকারের "ঠিক একবার পৌঁছানো" সাধারণভাবে অসম্ভব; কিন্তু At-least-once পৌঁছানো আর Idempotent প্রক্রিয়া—eventId, UNIQUE Constraint, Transaction—একসঙ্গে মিলে ফলাফল ঠিক একবারের মতো হয়।

ক্রম (Ordering): সব Event-এর একটি বিশ্বব্যাপী ক্রম ধরে নেবেন না; একাধিক Server, একাধিক সংযোগ ও Retry ক্রম এলোমেলো করে। যা দরকার তা হলো প্রতিটি Aggregate-এর (এখানে প্রতিটি Booking-এর) নিজস্ব `sequence`। Client শেষ দেখা sequence মনে রাখে, এবং প্রতিটি আসা Event-কে চার ভাগে ভাগ করে:

client-state.mjs · JavaScript — চলবে: Node.js

```
// Client-এর দিক: Duplicate বাদ, পুরোনো Event উপেক্ষা, ফাঁকে Snapshot চাই।
// Browser ও Node.js দুই জায়গাতেই চলে (কোনো Import নেই)।
export class BookingView {
  #state = new Map();
  #seen = new Set();

  constructor({ seenLimit = 500 } = {}) {
    this.seenLimit = seenLimit;
  }

  loadSnapshot({ bookingId, title, status, sequence }) {
    this.#state.set(bookingId, { title, status, sequence });
  }

  apply(event) {
    if (this.#seen.has(event.eventId)) return 'duplicate';
    const current = this.#state.get(event.bookingId);
    const last = current?.sequence ?? 0;
    if (event.sequence <= last) return 'stale';
    if (event.sequence !== last + 1) return 'gap';
    this.#remember(event.eventId);
```

```
    this.#state.set(event.bookingId, {
      title: current?.title ?? event.payload.title ?? '',
      status: event.payload.status,
      sequence: event.sequence,
    });
    return 'applied';
  }

  lastSequence(bookingId) {
    return this.#state.get(bookingId)?.sequence ?? 0;
  }

  get(bookingId) {
    return this.#state.get(bookingId);
  }

  #remember(eventId) {
    this.#seen.add(eventId);
    if (this.#seen.size > this.seenLimit) {
      this.#seen.delete(this.#seen.values().next().value);
    }
  }
}
```

নিয়মগুলো: আগে দেখা eventId হলে `duplicate`; sequence শেষ দেখাটির সমান বা কম হলে `stale` (পুরোনো); ঠিক পরেরটি হলে `applied`; আর মাঝে লাফ দিলে `gap` —কিছু হারিয়েছে, তাই এই Event প্রয়োগ না করে Server থেকে বাদ পড়াগুলো বা পূর্ণ Snapshot আনতে হবে। মনে রাখা eventId-এর Set সীমিত (`seenLimit`), নইলে দীর্ঘ সময় খোলা Tab-এর Memory বাড়তেই থাকবে; পুরোনো Duplicate তো sequence-এর নিয়মেই `stale` হিসেবে ধরা পড়ে।

dedupe-order.mjs · JavaScript চলবে: Node.js

```
import { BookingView } from '../project/travel-live/src/client-state.mjs';

const ev = (eventId, sequence, status) => ({ eventId, bookingId: 'bk-7',
  sequence, payload: { status } });

// Network-এ যা ঘটতে পারে: Retry-তে Duplicate, দেরিতে আসা, মাঝখানে হারানো।
const arrivals = [
  ev('e1', 1, 'PENDING'),
  ev('e2', 2, 'CONFIRMED'),
  ev('e2', 2, 'CONFIRMED'),
  ev('e4', 4, 'REFUNDED'),
  ev('e1', 1, 'PENDING'),
];
const view = new BookingView();
for (const event of arrivals) {
  console.log(`${event.eventId} (#${event.sequence}):`, view.apply(event));
}
```

```
console.log('দেখানো অবস্থা:', view.get('bk-7'));

// ফাঁক (#৩ নেই): Server থেকে Snapshot এনে তারপর এগোনো।
view.loadSnapshot({ bookingId: 'bk-7', title: 'Sajek', status: 'CANCELLED',
  sequence: 3 });
console.log('Snapshot-এর পরে #4:', view.apply(ev('e4', 4, 'REFUNDED')));
console.log('দেখানো অবস্থা:', view.get('bk-7'));
```

Output

```
e1 (#1): applied
e2 (#2): applied
e2 (#2): duplicate
e4 (#4): gap
e1 (#1): duplicate
দেখানো অবস্থা: { title: '', status: 'CONFIRMED', sequence: 2 }
Snapshot-এর পরে #4: applied
দেখানো অবস্থা: { title: 'Sajek', status: 'REFUNDED', sequence: 4 }
```

এখানে অবস্থার নামগুলো (যেমন REFUNDED) শুধু উদাহরণ; `BookingView` কোনো নির্দিষ্ট অবস্থা-যন্ত্র জানে না, সে শুধু ক্রম রক্ষা করে। #৪ প্রথমবার এসে ফাঁক দেখায়, তাই Screen #২-তেই থাকে—ভুল অবস্থা দেখানোর চেয়ে কিছুক্ষণ পুরোনো অবস্থা দেখানো ভালো। Snapshot আসার পর একই Event নির্বিঘ্নে প্রয়োগ হয়।

২৭.৯ Reconnect, Heartbeat ও Backpressure

পুনঃসংযোগ: সংযোগ কাটা স্বাভাবিক ঘটনা—Wi-Fi বদল, Laptop ঘুম, Server Deploy। সঙ্গে সঙ্গে, বারবার চেষ্টা করলে বিপদ: Server Restart-এর পর হাজার Client একই মুহূর্তে ফিরে এসে নতুন Server-কেও ফেলে দেয় (**Thundering Herd**)। সমাধান: প্রতি ব্যর্থতায় অপেক্ষা দ্বিগুণ (**Exponential Backoff**), একটি সর্বোচ্চ সীমা, এবং কিছুটা এলোমেলো (**Jitter**) যাতে Client-রা ছড়িয়ে পড়ে।

backoff.mjs · JavaScript চলবে: Node.js

```
// Exponential Backoff + Jitter: প্রতিবার অপেক্ষা দ্বিগুণ, সর্বোচ্চ ৩০ সেকেন্ড,
// আর ±২৫% এলোমেলো, যাতে হাজার Client একসঙ্গে ফিরে না আসে।
export function reconnectDelay(attempt, random = Math.random) {
  const capped = Math.min(30_000, 500 * 2 ** Math.min(attempt, 6));
  return Math.floor(capped * (0.75 + random() * 0.5));
}
```

`random` একটি Parameter, যাতে Test-এ নির্দিষ্ট মান দেওয়া যায়:

backoff-demo.mjs · JavaScript চলবে: Node.js

```
import { reconnectDelay } from '../project/travel-live/src/backoff.mjs';
```

```
// Test-যোগ্য করতে random বাইরে থেকে দেওয়া: 0.5 মানে Jitter শূন্য।
const middle = () => 0.5;
for (const attempt of [0, 1, 2, 3, 4, 5, 6, 10]) {
  console.log(`চেষ্টা ${attempt}: ${reconnectDelay(attempt, middle)} ms`);
}

// আসল Jitter: ১০০০ Client একসঙ্গে বিচ্ছিন্ন হলে ৩য় চেষ্টা কতটা ছড়ায়?
let seed = 27;
const random = () => {
  seed = (seed * 1_103_515_245 + 12_345) % 2_147_483_648;
  return seed / 2_147_483_648;
};
const delays = Array.from({ length: 1000 }, () => reconnectDelay(3, random));
console.log('৩য় চেষ্টা: সর্বনিম্ন', Math.min(...delays), 'ms, সর্বোচ্চ',
  Math.max(...delays), 'ms');
```

Output

```
চেষ্টা 0: 500 ms
চেষ্টা 1: 1000 ms
চেষ্টা 2: 2000 ms
চেষ্টা 3: 4000 ms
চেষ্টা 4: 8000 ms
চেষ্টা 5: 16000 ms
চেষ্টা 6: 30000 ms
চেষ্টা 10: 30000 ms
৩য় চেষ্টা: সর্বনিম্ন 3000 ms, সর্বোচ্চ 4997 ms
```

চেষ্টা ৬-এ হিসাব ৩২ সেকেন্ড হতো, কিন্তু সীমা ৩০ সেকেন্ড। আর ১০০০ Client-এর তৃতীয় চেষ্টা একই মুহূর্তে নয়, তিন থেকে পাঁচ সেকেন্ডে ছড়িয়ে। সফল সংযোগের পর `attempt` আবার শূন্য করতে ভুলবেন না।

Heartbeat: TCP সংযোগ কখনো নিঃশব্দে মরে যায়—Mobile Network বদলালে বা একটি NAT Router সংযোগ ভুলে গেলে কেউ Close Frame পাঠায় না। Server ভাবে Client আছে, আর তার জন্য Memory ও Subscription ধরে রাখে। WebSocket Protocol-এ **Ping** ও **Pong** Frame আছে: Server নির্দিষ্ট বিরতিতে Ping পাঠায়; পরের Ping-এর আগে Pong না এলে সংযোগ মৃত ধরে বন্ধ করে। Browser Pong স্বয়ংক্রিয়ভাবে পাঠায়, তাই Client-এ কিছু লিখতে হয় না। SSE-তে একই কাজ করে একটি মন্তব্য Line (`: ping`), যা মাঝের Proxy-কে জানায় সংযোগটি অলস নয়।

Backpressure: Server যত দ্রুত Message তৈরি করে, একটি ধীর Client (দুর্বল Mobile Network) তত দ্রুত নিতে পারে না। পাঠানো কিন্তু না-পৌঁছানো Data Server-এর Buffer-এ জমে (`ws` -এ `bufferedAmount`)। সীমাহীন Queue মানে একটি ধীর Client-ই পুরো Server-এর Memory শেষ করতে পারে। তিনটি প্রচলিত নীতি:

backpressure.mjs · JavaScript চলবে: Node.js

```
// ধীর Client-এর জন্য সীমিত Queue। তিনটি নীতি: পুরোনো ফেলা, একীকরণ, বিচ্ছিন্ন।
function createQueue({ limit, policy }) {
  const items = [];
  return {
```

```
    items,
    push(event) {
      if (policy === 'coalesce') {
        // একই Booking-এর পুরোনো Event থাকলে শুধু সর্বশেষটি রাখা।
        const i = items.findIndex((e) => e.bookingId === event.bookingId);
        if (i !== -1) items.splice(i, 1);
      }
      if (items.length < limit) return items.push(event) && 'queued';
      if (policy === 'drop-oldest' || policy === 'coalesce') {
        items.shift();
        items.push(event);
        return 'dropped-oldest';
      }
      return 'disconnect';
    },
  };
}

const burst = [1, 2, 3, 4, 5, 6].map((n) => ({
  bookingId: `bk-${n % 2 === 0 ? 'A' : 'B'}`, sequence: n }));
for (const policy of ['drop-oldest', 'coalesce', 'disconnect']) {
  const queue = createQueue({ limit: 3, policy });
  const results = burst.map((event) => queue.push(event));
  console.log(policy.padEnd(11), results.at(-1).padEnd(14),
    queue.items.map((e) => `${e.bookingId}#${e.sequence}`).join(' '));
}
```

Output

```
drop-oldest dropped-oldest bk-A#4 bk-B#5 bk-A#6
coalesce    queued         bk-B#5 bk-A#6
disconnect  disconnect     bk-B#1 bk-A#2 bk-B#3
```

`drop-oldest` পুরোনো ফেলে দেয়—Client-এ ফাঁক হবে, তাই তাকে Snapshot চাইতে হবে। `coalesce` একই Booking-এর পুরোনো Event-এর জায়গায় শুধু সর্বশেষটি রাখে—অবস্থা দেখানোর Dashboard-এর জন্য চমৎকার, কিন্তু প্রতিটি Event জরুরি হলে (যেমন Chat) চলবে না। `disconnect` ধীর Client-কে বিচ্ছিন্ন করে; Client পুনঃসংযোগে Replay চাইবে। TravelLive তৃতীয় নীতি নেয়, কারণ তার Replay ব্যবস্থা ইতিমধ্যে আছে।

২৭.১০ একাধিক Server: Pub/Sub ও Scaling

একটি Server-এ হাজার কয়েক সংযোগ সহজেই চলে। কিন্তু Load Balancer-এর পেছনে দুটি Instance রাখলে নতুন সমস্যা: রাফির Phone Instance A-তে, নাদিয়ার Laptop Instance B-তে। নাদিয়ার Command B-তে এলে A-র Client কীভাবে জানবে? উত্তর: একটি **Broker**। প্রতিটি Instance প্রাসঙ্গিক Channel-এ Subscribe করে; যেকোনো Instance

Publish করলে সবাই পায় এবং নিজের Client-দের পাঠায়। Redis Pub/Sub এর একটি জনপ্রিয় সমাধান, এবং Socket.IO-র Redis Adapter ভেতরে এটিই ব্যবহার করে।

QA-যন্ত্রে Redis ছিল না, তাই নিচের Broker একটি ছোট in-memory অনুকরণ—আসল Redis নয়—কিন্তু Redis Pub/Sub-এর মূল আচরণ হুবহু দেখায়:

pubsub-fanout.mjs · JavaScript চলবে: Node.js

```
// দুটি Gateway Instance, মাঝে একটি Broker। এখানে Broker একটি সাধারণ
// in-memory অনুকরণ, যা Redis Pub/Sub-এর মূল আচরণ দেখায়; আসল Redis নয়।
class PubSub {
  #channels = new Map();
  subscribe(channel, fn) {
    const set = this.#channels.get(channel) ?? new Set();
    set.add(fn);
    this.#channels.set(channel, set);
    return () => set.delete(fn);
  }
  publish(channel, message) {
    // শুধু এই মুহূর্তের Subscriber পায়; কোনো History জমা থাকে না।
    for (const fn of this.#channels.get(channel) ?? []) fn(message);
  }
}

function gateway(name, broker) {
  const local = new Map(); // এই Instance-এ যুক্ত Client
  return {
    connect(client, room) {
      local.set(client, room);
      return broker.subscribe(room, (msg) => {
        if (local.get(client) === room) console.log(`  ${name}/${client}:`, msg);
      });
    },
    publish: (room, msg) => broker.publish(room, msg),
  };
}

const broker = new PubSub();
const a = gateway('A', broker);
const b = gateway('B', broker);
a.connect('rafi-phone', 'booking:bk-1');
const leave = b.connect('agent-laptop', 'booking:bk-1');

console.log('Instance A থেকে Publish:');
a.publish('booking:bk-1', { sequence: 2, status: 'CONFIRMED' });
leave();
console.log('Agent offline থাকাকালে Publish:');
b.publish('booking:bk-1', { sequence: 3, status: 'CANCELLED' });
```

```
b.connect('agent-laptop', 'booking:bk-1');
console.log('Agent ফিরে এল; #3 আর আসবে না, Snapshot/Replay লাগবে');
```

Output

```
Instance A থেকে Publish:
  A/rafi-phone: { sequence: 2, status: 'CONFIRMED' }
  B/agent-laptop: { sequence: 2, status: 'CONFIRMED' }
Agent offline থাকাকালে Publish:
  A/rafi-phone: { sequence: 3, status: 'CANCELLED' }
Agent ফিরে এল; #3 আর আসবে না, Snapshot/Replay লাগবে
```

শেষ Line-টিই সবচেয়ে গুরুত্বপূর্ণ: **Redis Pub/Sub একটি টেকসই Queue নয়।** Message শুধু সেই মুহূর্তে সংযুক্ত Subscriber পায়; কেউ না থাকলে তা চিরতরে হারায়। তাই নির্ভরযোগ্য Replay-র জন্য তিনটির একটি লাগে: Database-এ Event History (TravelLive-এর `history` -র মতো, কিন্তু টেকসই); Redis Streams, Kafka বা NATS JetStream-এর মতো টেকসই Log, যেখানে Consumer নিজের অবস্থান মনে রাখে; অথবা অন্তত পুনঃসংযোগে Database থেকে পূর্ণ Snapshot।

আরও দুটি বাস্তব বিষয়। **Sticky Session:** Socket.IO-র Long Polling Fallback একই Instance-এ ফিরে আসা দরকার, তাই Load Balancer-এ Sticky Session চালু করতে হয়; খাঁটি WebSocket-এ সংযোগ নিজেই এক Instance-এ থাকে। **sequence কে দেবে?** কখনো Gateway নয়—দুটি Gateway আলাদা গুনবে। sequence আসবে Booking-এর মালিক Service থেকে, Database Transaction-এর ভেতরে, Outbox সারির সঙ্গে।

২৭.১১ নিরাপত্তা Checklist

১. **TLS:** Production-এ শুধু `wss://` ও `https://`।
২. **Origin:** WebSocket Upgrade-এ `Origin` Header একটি অনুমোদিত তালিকার সঙ্গে মেলান। এটি Cross-Site WebSocket Hijacking ঠেকায়, যেখানে একটি অচেনা Site আপনার ব্যবহারকারীর Cookie দিয়ে সংযোগ খোলে।
৩. **Authentication:** সংযোগের শুরুতে, অল্প সময়ের মধ্যে; না হলে বন্ধ। Token URL-এ নয়।
৪. **Authorization প্রতিবার:** প্রতিটি Subscribe-এ, এবং প্রতিটি Event পাঠানোর সময়ও—কারণ এর মধ্যে অনুমতি বাতিল হতে পারে (Logout, ভূমিকা বদল)।
৫. **Input:** Message-এর আকারের সীমা, JSON-এর ধরন যাচাই, প্রতি সংযোগে হারের সীমা, প্রতি সংযোগে Subscription-এর সংখ্যার সীমা।
৬. **তথ্য কমানো:** Event-এ শুধু যা দরকার; অস্তিত্ব ফাঁস না করতে অজানা ও নিষিদ্ধ Booking-এ একই উত্তর।
৭. **পর্যবেক্ষণ:** সক্রিয় সংযোগ, প্রত্যাখ্যাত Origin, Auth ব্যর্থতা, ধীর Client-এর বিচ্ছিন্নতা ও পুনঃসংযোগের হার মাপুন—হঠাৎ বাড়লে কিছু ভুল হচ্ছে।

২৭.১২ Project: TravelLive

TravelLive একটি Real-Time Booking Dashboard। গ্রাহক নিজের Booking-এর অবস্থা Live দেখেন; Agent সব Booking দেখেন এবং নিশ্চিত বা বাতিল করেন। এটি v30-এর TravelLive Simulator-এর সব নিয়ম রাখে—অবস্থা-যন্ত্র, Room-এর অনুমতি, sequence-ভিত্তিক Replay, Duplicate বাদ, সীমিত History, Backoff—এবং তার ওপর আসল Network যোগ করে: `ws` দিয়ে WebSocket Gateway, `node:http` দিয়ে SSE ও JSON API, এবং একটি Browser Dashboard।

ব্যবসার নিয়ম: `PENDING` থেকে `CONFIRMED` বা `CANCELLED`; `CONFIRMED` থেকে `CANCELLED`; `CANCELLED` শেষ অবস্থা। শুধু Agent নিশ্চিত করতে পারেন; মালিক ও Agent বাতিল করতে পারেন। গ্রাহক শুধু নিজের Booking দেখেন।

File-এর কাঠামো

Folder-এর কাঠামো

```
travel-live/
├── package.json          (একটি Dependency: ws)
├── README.md
├── main.mjs              Server চালু, SIGTERM-এ সুশৃঙ্খল বন্ধ
├── demo.mjs              এক Process-এ পুরো প্রবাহ
├── public/
│   ├── index.html        Browser Dashboard
│   └── app.mjs           WebSocket Client, Backoff, Replay
├── src/
│   ├── events.mjs        Event Envelope ও যাচাই
│   ├── live-hub.mjs      Domain: Booking, অনুমতি, History, Subscription
│   ├── client-state.mjs  Duplicate, পুরোনো ও ফাঁক চেনা
│   ├── backoff.mjs       Exponential Backoff + Jitter
│   ├── rate-limit.mjs    Token Bucket
│   ├── gateway.mjs       WebSocket: Origin, Auth, Protocol, Heartbeat
│   ├── sse.mjs           SSE Stream ও Last-Event-ID
│   ├── server.mjs        HTTP Route ও Static File
│   └── demo-data.mjs     Demo ব্যবহারকারী ও Booking
└── tests/                ১৮টি Test (Domain, Client, HTTP, SSE, WebSocket)
```

Domain (`live-hub.mjs`) কোনো Network জানে না; Gateway ও SSE দুটিই একই Hub ব্যবহার করে। এই বিভাজনের ফলে Domain-এর নিয়ম Network ছাড়াই Test করা যায়, আর ভবিষ্যতে Socket.IO বা অন্য Transport যোগ করলে Domain বদলায় না।

src/live-hub.mjs

প্রথম অংশ: অবস্থা-যন্ত্র (কোন অবস্থা থেকে কোথায় যাওয়া যায়), নিজস্ব Error, Private Field-এ রাখা অবস্থা, এবং ব্যবহারকারী যোগ ও বাতিল:

live-hub.mjs · JavaScript অংশ ১ / 8 (Part 1 of 4) চলবে: Node.js

```
import { createEvent } from './events.mjs';

// Booking-এর অবস্থা বদলের নিয়ম। CANCELLED শেষ অবস্থা।
const TRANSITIONS = {
  PENDING: ['CONFIRMED', 'CANCELLED'],
  CONFIRMED: ['CANCELLED'],
  CANCELLED: [],
};

export class LiveError extends Error {
  constructor(code, message) {
    super(message);
    this.code = code;
  }
}

export class LiveHub {
  #bookings = new Map();
  #history = new Map();
  #subscribers = new Map();
  #users = new Map();
  #counter = 0;

  constructor({ historyLimit = 50, now = () => new Date() } = {}) {
    this.historyLimit = historyLimit;
    this.now = now;
  }

  addUser(user) {
    this.#users.set(user.id, { ...user, active: true });
  }

  revokeUser(userId) {
    const user = this.#users.get(userId);
    if (user) user.active = false;
    this.#recheck();
  }

// ...
```

`canView` একটি জায়গায় সিদ্ধান্ত নেয়—সক্রিয় ব্যবহারকারী, এবং Agent অথবা মালিক। তারপর নতুন Booking ও অবস্থা-বদল। ID ও মালিক Server ঠিক করে; Client কখনো নয়:

live-hub.mjs · JavaScript অংশ ২ / 8 (Part 2 of 4) চলবে: Node.js

```
// ...
```

```
  canView(userId, bookingId) {
    const user = this.#users.get(userId);
    const booking = this.#bookings.get(bookingId);
    if (!user?.active || !booking) return false;
    return user.role === 'agent' || booking.ownerId === userId;
  }

  createBooking(actorId, title) {
    if (!this.#users.get(actorId)?.active) {
      throw new LiveError('UNAUTHORIZED', 'অজানা বা বাতিল ব্যবহারকারী');
    }
    const id = `bk-${this.#bookings.size + 1}`;
    const booking = { id, ownerId: actorId, title, status: 'PENDING',
      sequence: 0 };
    this.#bookings.set(id, booking);
    this.#history.set(id, []);
    return this.#record(booking, 'booking.created',
      { status: 'PENDING', title });
  }

  changeStatus(actorId, bookingId, status) {
    if (!this.canView(actorId, bookingId)) {
      throw new LiveError('FORBIDDEN', 'এই Booking বদলানোর অনুমতি নেই');
    }
    const booking = this.#bookings.get(bookingId);
    const actor = this.#users.get(actorId);
    if (status === 'CONFIRMED' && actor.role !== 'agent') {
      throw new LiveError('FORBIDDEN', 'শুধু Agent নিশ্চিত করতে পারেন');
    }
    if (!TRANSITIONS[booking.status].includes(status)) {
      throw new LiveError('INVALID_TRANSITION',
        `${booking.status} থেকে ${status} সম্ভব নয়`);
    }
    const from = booking.status;
    booking.status = status;
    return this.#record(booking, 'booking.status.changed', { from, status });
  }

// ...
```

`changeStatus` তিনটি জিনিস যাচাই করে—দেখার অনুমতি, ভূমিকা, এবং বৈধ অবস্থা-বদল—তারপর অবস্থা বদলায় ও Event রেকর্ড করে। এরপর Snapshot ও Subscribe:

live-hub.mjs · JavaScript অংশ ৩ / 8 (Part 3 of 4) চলবে: Node.js

```
// ...

  snapshot(userId, bookingId) {
    if (!this.canView(userId, bookingId)) {
```

```
      throw new LiveError('FORBIDDEN', 'এই Booking দেখার অনুমতি নেই');
    }
    const { id, title, status, sequence } = this.#bookings.get(bookingId);
    return { bookingId: id, title, status, sequence };
  }

  // Subscribe-এর সময় অনুমতি যাচাই; afterSequence-এর পরের Event Replay,
  // আর History-তে ফাঁক থাকলে পূর্ণ Snapshot।
  subscribe(userId, bookingId, listener, { afterSequence = 0,
    onClose = () => {} } = {}) {
    if (!this.canView(userId, bookingId)) {
      throw new LiveError('FORBIDDEN', 'এই Booking দেখার অনুমতি নেই');
    }
    const sub = { userId, listener, onClose };
    const subs = this.#subscribers.get(bookingId) ?? new Set();
    subs.add(sub);
    this.#subscribers.set(bookingId, subs);
    const history = this.#history.get(bookingId);
    const oldest = history[0]?.sequence ?? 1;
    const recovery = afterSequence + 1 < oldest
      ? { mode: 'snapshot', snapshot: this.snapshot(userId, bookingId) }
      : { mode: 'replay',
        events: history.filter((e) => e.sequence > afterSequence) };
    return { ...recovery, unsubscribe: () => subs.delete(sub) };
  }

// ...
```

Subscribe-এর উত্তর দুই রকম: History-তে প্রয়োজনীয় সব Event থাকলে `replay` (শুধু `afterSequence`-এর পরেরগুলো), আর History সীমিত বলে পুরোনোগুলো মুছে গেলে `snapshot`। শেষ অংশ:

live-hub.mjs · JavaScript অংশ 8 / 8 (Part 4 of 4) চলবে: Node.js

```
// ...

  subscriberCount(bookingId) {
    return this.#subscribers.get(bookingId)?.size ?? 0;
  }

  #record(booking, type, payload) {
    booking.sequence += 1;
    this.#counter += 1;
    const event = createEvent({ id: `evt-${this.#counter}`, type,
      bookingId: booking.id, sequence: booking.sequence, payload,
      now: this.now() });
    const history = this.#history.get(booking.id);
    history.push(event);
    if (history.length > this.historyLimit) history.shift();
    this.#dispatch(event);
```

```
    return event;
  }

  // Dispatch-এর সময়ও অনুমতি আবার যাচাই: বাতিল ব্যবহারকারী কিছু পান না।
  #dispatch(event) {
    for (const sub of this.#subscribers.get(event.bookingId) ?? []) {
      if (this.canView(sub.userId, event.bookingId)) sub.listener(event);
      else this.#close(event.bookingId, sub);
    }
  }

  #recheck() {
    for (const [bookingId, subs] of this.#subscribers) {
      for (const sub of subs) {
        if (!this.canView(sub.userId, bookingId)) this.#close(bookingId, sub);
      }
    }
  }

  #close(bookingId, sub) {
    this.#subscribers.get(bookingId).delete(sub);
    sub.onClose('FORBIDDEN');
  }
}
```

`#record` sequence বাড়ায়, Event তৈরি করে, History-তে রাখে (সীমা ছাড়ালে সবচেয়ে পুরোনোটি বাদ) এবং পাঠায়। `#dispatch` প্রতিটি Subscriber-কে পাঠানোর **ঠিক আগে** আবার `canView` যাচাই করে: অনুমতি বাতিল হলে Subscription বন্ধ হয় এবং `onClose('FORBIDDEN')` জানানো হয়। `revokeUser` সঙ্গে সঙ্গে `#recheck` চালায়, তাই পরের Event-এর অপেক্ষাও করতে হয় না।

> **নোট:** এই Hub-এ অবস্থা-বদল ও History-তে লেখা একই Synchronous ধাপে ঘটে, তাই একটি Process-এর মধ্যে তারা কখনো আলাদা হয় না। একাধিক Process-এ এই নিশ্চয়তা পেতে Database Transaction ও Outbox লাগবে (অনুচ্ছেদ ২৭.৭)।

src/gateway.mjs

Gateway HTTP Server-এর `upgrade` Event ধরে। প্রথম কাজ: Path ও Origin যাচাই, তারপর `ws` -কে Handshake করতে দেওয়া। `maxPayload` 8 KB-র বড় Message-এ সংযোগ 1009 দিয়ে বন্ধ করে:

gateway.mjs · JavaScript অংশ ১ / 8 (Part 1 of 4) চলবে: Node.js

```
import WebSocket from 'ws';
import { createBucket } from './rate-limit.mjs';

const CLOSE = { AUTH: 4401, FORBIDDEN_ORIGIN: 4403, RATE: 4429 };
```

```
// HTTP Server-এর 'upgrade' Event-এ WebSocket Gateway যুক্ত করে।
export function attachGateway(server, { hub, users, allowedOrigins,
  heartbeatMs = 30_000, authTimeoutMs = 3_000, maxBuffered = 64 * 1024 }) {
  const wss = new WebSocket.Server({ noServer: true, maxPayload: 4 * 1024 });
  const metrics = { connections: 0, rejectedOrigins: 0, slowConsumers: 0,
    protocolErrors: 0 };

  server.on('upgrade', (req, socket, head) => {
    const { pathname } = new URL(req.url, 'http://localhost');
    const origin = req.headers.origin;
    if (pathname !== '/live') return socket.destroy();
    // Browser সব সময় Origin পাঠায়; অচেনা Site-এর Page-কে ঢুকতে দেবেন না।
    if (origin && !allowedOrigins.includes(origin)) {
      metrics.rejectedOrigins += 1;
      socket.end('HTTP/1.1 403 Forbidden\r\n\r\n');
      return undefined;
    }
    return wss.handleUpgrade(req, socket, head,
      (ws) => wss.emit('connection', ws));
  });

// ...
```

প্রতিটি সংযোগের নিজস্ব অবস্থা: ব্যবহারকারী (Auth-এর আগে `null`), Subscription-এর তালিকা, এবং একটি Token Bucket (একসঙ্গে ২০টি, তারপর সেকেন্ডে ৫টি Message)। Auth-এর জন্য ৩ সেকেন্ড সময়। শেষে `send` : Buffer সীমা ছাড়ালে ধীর Client বিচ্ছিন্ন:

gateway.mjs · JavaScript অংশ ২ / 8 (Part 2 of 4) চলবে: Node.js

```
// ...

  wss.on('connection', (ws) => {
    metrics.connections += 1;
    const conn = { ws, user: null, subs: new Map(),
      take: createBucket({ capacity: 20, refillPerSecond: 5 }) };
    const authTimer = setTimeout(
      () => ws.close(CLOSE.AUTH, 'auth timeout'), authTimeoutMs);
    ws.on('message', (data) => {
      if (!conn.take()) return ws.close(CLOSE.RATE, 'rate limit');
      return handleMessage(conn, String(data), { hub, users, authTimer,
        send: (msg) => send(conn, msg) });
    });
    // ভুল Frame বা সীমা-ছাড়ানো Message-এ 'error'; ws নিজেই পরে বন্ধ করে।
    // Handler না থাকলে পুরো Process ভেঙে পড়ত।
    ws.on('error', () => { metrics.protocolErrors += 1; });
    ws.on('close', () => {
      clearTimeout(authTimer);
      for (const unsubscribe of conn.subs.values()) unsubscribe();
      metrics.connections -= 1;
```

```
    });
  });

  // Slow Client: Buffer সীমা ছাড়ালে Connection বন্ধ; Client পরে Replay চাইবে।
  function send(conn, message) {
    if (conn.ws.readyState !== WebSocket.OPEN) return;
    if (conn.ws.bufferedAmount > maxBuffered) {
      metrics.slowConsumers += 1;
      conn.ws.terminate();
      return;
    }
    conn.ws.send(JSON.stringify(message));
  }

// ...
```

`ws.on('error')` Handler ছোট, কিন্তু অপরিহার্য। Test লেখার সময় দেখা গেছে: এটি না থাকলে একটি বড় Message পাঠিয়েই যে কেউ পুরো Server Process ভেঙে দিতে পারত, কারণ `ws` সীমা-ছাড়ানো Message-এ সংযোগে একটি `error` Event তোলে, আর Node.js-এ Handler-হীন `error` Event পুরো Process থামিয়ে দেয়। পরের অংশ: Heartbeat, এবং Protocol-এর প্রথম ধাপ `handleMessage`। Auth-এর আগে অন্য যেকোনো Message-এ সংযোগ বন্ধ:

gateway.mjs · JavaScript অংশ ৩ / 8 (Part 3 of 4) চলবে: Node.js

```
// ...

  // Heartbeat: আগের Ping-এর Pong না এলে Connection মৃত ধরে নিন।
  const timer = setInterval(() => {
    for (const ws of wss.clients) {
      if (ws.isAlive === false) { ws.terminate(); continue; }
      ws.isAlive = false;
      ws.ping();
    }
  }, heartbeatMs);
  wss.on('connection', (ws) => {
    ws.isAlive = true;
    ws.on('pong', () => { ws.isAlive = true; });
  });
  server.on('close', () => clearInterval(timer));

  return { wss, metrics };
}

function handleMessage(conn, text, { hub, users, authTimer, send }) {
  let msg;
  try {
    msg = JSON.parse(text);
  } catch {
```

```
    return send({ type: 'error', code: 'BAD_JSON' });
  }
  if (msg.type === 'auth') {
    const user = users.get(msg.token);
    if (!user) return conn.ws.close(CLOSE.AUTH, 'invalid token');
    clearTimeout(authTimer);
    conn.user = user;
    return send({ type: 'auth.ok', userId: user.id });
  }
  if (!conn.user) return conn.ws.close(CLOSE.AUTH, 'auth required');
  if (msg.type === 'subscribe') return subscribe(conn, msg, hub, send);
  if (msg.type === 'unsubscribe') {
    conn.subs.get(msg.bookingId)?.();
    conn.subs.delete(msg.bookingId);
    return send({ type: 'ack', requestId: msg.requestId, ok: true });
  }
  return send({ type: 'error', code: 'UNKNOWN_TYPE' });
}

// ...
```

শেষ অংশ `subscribe`। প্রতিটি অনুরোধের উত্তরে `requestId`-সহ একটি `ack`, যাতে Client জানে কোন অনুরোধের ফল কী—Socket.IO-র Acknowledgement-এর সমতুল্য:

gateway.mjs · JavaScript অংশ 8 / 8 (Part 4 of 4) চলবে: Node.js

```
// ...

function subscribe(conn, msg, hub, send) {
  const { bookingId, requestId } = msg;
  const afterSequence = Number.isInteger(msg.afterSequence)
    ? msg.afterSequence : 0;
  if (typeof bookingId !== 'string' || conn.subs.has(bookingId)) {
    return send({ type: 'ack', requestId, ok: false, code: 'BAD_REQUEST' });
  }
  try {
    const result = hub.subscribe(conn.user.id, bookingId,
      (event) => send({ type: 'event', event }),
      { afterSequence, onClose: (reason) => {
        conn.subs.delete(bookingId);
        send({ type: 'unsubscribed', bookingId, reason });
      } });
    conn.subs.set(bookingId, result.unsubscribe);
    send({ type: 'ack', requestId, ok: true, mode: result.mode });
    if (result.mode === 'snapshot') {
      return send({ type: 'snapshot', snapshot: result.snapshot });
    }
    for (const event of result.events) send({ type: 'event', event });
    return undefined;
```

```
  } catch (error) {
    return send({ type: 'ack', requestId, ok: false, code: error.code });
  }
}
```

লক্ষ করুন, `subscribe`-এ প্রথমে `ack`, তারপর Replay-র Event বা Snapshot। Hub-এর `subscribe` Subscriber-কে নিবন্ধিত করে Replay ফেরত দেয় একই Synchronous ধাপে, তাই Replay ও Live Event-এর মাঝে কোনো Event হারানোর ফাঁক নেই।

src/sse.mjs

একই Hub, ভিন্ন Transport। Browser-এর `Last-Event-ID` থেকে `afterSequence`:

sse.mjs · JavaScript — চলবে: Node.js

```
// Server-Sent Events: একমুখী (Server থেকে Client) সাধারণ HTTP Stream।
// Browser পুনঃসংযোগে নিজেই Last-Event-ID পাঠায়; তা দিয়ে Replay।
export function streamBooking(req, res, { hub, user, bookingId,
  heartbeatMs = 15_000 }) {
  const lastId = Number.parseInt(req.headers['last-event-id'] ?? '0', 10);
  const write = (event) => res.write(
    `id: ${event.sequence}\nevent: ${event.type}\n`
    + `data: ${JSON.stringify(event)}\n\n`);
  const result = hub.subscribe(user.id, bookingId, write, {
    afterSequence: Number.isInteger(lastId) ? lastId : 0,
    onClose: () => res.end(),
  });
  res.writeHead(200, {
    'Content-Type': 'text/event-stream; charset=utf-8',
    'Cache-Control': 'no-cache',
    Connection: 'keep-alive',
  });
  res.write('retry: 3000\n\n');
  if (result.mode === 'snapshot') {
    res.write(`event: snapshot\ndata: ${JSON.stringify(result.snapshot)}\n\n`);
  } else {
    result.events.forEach(write);
  }
  // Comment Line (':' দিয়ে শুরু) Proxy-কে জানায় Connection জীবিত।
  const timer = setInterval(() => res.write(': ping\n\n'), heartbeatMs);
  req.on('close', () => {
    clearInterval(timer);
    result.unsubscribe();
  });
}
```

Response-এর Header লেখার আগে `hub.subscribe` ডাকা হয়, যাতে অনুমতি না থাকলে Error সাধারণ JSON 403 হিসেবে যায়। Client চলে গেলে (`req` 'close') Timer ও Subscription দুটিই পরিষ্কার—নইলে প্রতিটি বন্ধ Tab একটি Memory Leak রেখে যেত।

src/server.mjs

HTTP Route, JSON Body-র সীমা ও Error চুক্তি—অধ্যায় ২৫-এর মতোই `{ data }` অথবা `{ error: { code, message } }`:

server.mjs · JavaScript অংশ ১ / 8 (Part 1 of 4) চলবে: Node.js

```
import http from 'node:http';
import { readFile } from 'node:fs/promises';
import { attachGateway } from './gateway.mjs';
import { streamBooking } from './sse.mjs';

const STATUS = { UNAUTHORIZED: 401, FORBIDDEN: 403, INVALID_TRANSITION: 409,
  BAD_REQUEST: 400, TOO_LARGE: 413, NOT_FOUND: 404 };
const STATIC = {
  '/': ['../public/index.html', 'text/html'],
  '/app.mjs': ['../public/app.mjs', 'text/javascript'],
  '/client-state.mjs': ['./client-state.mjs', 'text/javascript'],
  '/backoff.mjs': ['./backoff.mjs', 'text/javascript'],
};

class HttpError extends Error {
  constructor(code, message) {
    super(message);
    this.code = code;
  }
}

function json(res, status, body) {
  res.writeHead(status, { 'Content-Type': 'application/json; charset=utf-8' });
  res.end(JSON.stringify(body));
}

async function readJson(req, limit = 1024) {
  let size = 0;
  const chunks = [];
  for await (const chunk of req) {
    size += chunk.length;
    if (size > limit) throw new HttpError('TOO_LARGE', 'Body খুব বড়');
    chunks.push(chunk);
  }
  try {
    return JSON.parse(Buffer.concat(chunks).toString('utf8') || '{}');
  } catch {
    throw new HttpError('BAD_REQUEST', 'অবৈধ JSON');
```

```
  }
}

// ...
```

Bearer Token থেকে ব্যবহারকারী, এবং প্রতিটি অপ্রত্যাশিত Error-এ 500—ভেতরের বার্তা Client-কে নয়, Log-এ:

server.mjs · JavaScript অংশ ২ / 8 (Part 2 of 4) চলবে: Node.js

```
// ...

export function createLiveServer({ hub, users, allowedOrigins,
  heartbeatMs, authTimeoutMs }) {
  const authenticate = (req) => {
    const token = /^Bearer (.+)$/.exec(req.headers.authorization ?? '')?.[1];
    const user = users.get(token);
    if (!user) throw new HttpError('UNAUTHORIZED', 'Token প্রয়োজন');
    return user;
  };

  const server = http.createServer(async (req, res) => {
    try {
      await route(req, res);
    } catch (error) {
      const status = STATUS[error.code];
      if (!status) console.error(error);
      json(res, status ?? 500, { error: { code: error.code ?? 'INTERNAL',
        message: status ? error.message : 'Server-এ সমস্যা' } });
    }
  });

// ...
```

Route-গুলো:

server.mjs · JavaScript অংশ ৩ / 8 (Part 3 of 4) চলবে: Node.js

```
// ...

  async function route(req, res) {
    const { pathname } = new URL(req.url, 'http://localhost');
    const [, api, resource, id, action] = pathname.split('/');
    if (req.method === 'GET' && STATIC[pathname]) {
      const [file, type] = STATIC[pathname];
      const body = await readFile(new URL(file, import.meta.url));
      res.writeHead(200, { 'Content-Type': `${type}; charset=utf-8` });
      return res.end(body);
    }
    if (req.method === 'GET' && pathname === '/health') {
```

```
      return json(res, 200, { data: { status: 'ok' } });
    }
    if (api !== 'api' || resource !== 'bookings') {
      throw new HttpError('NOT_FOUND', 'Route পাওয়া যায়নি');
    }
    const user = authenticate(req);
    if (req.method === 'POST' && !id) {
      const { title } = await readJson(req);
      if (typeof title !== 'string' || !title.trim()) {
        throw new HttpError('BAD_REQUEST', 'title প্রয়োজন');
      }
      const event = hub.createBooking(user.id, title.trim());
      return json(res, 201, { data: hub.snapshot(user.id, event.bookingId) });
    }
    if (req.method === 'POST' && action === 'status') {
      const { status } = await readJson(req);
      const event = hub.changeStatus(user.id, id, status);
      return json(res, 200, { data: event });
    }
    if (req.method === 'GET' && id && action === 'stream') {
      if (!hub.canView(user.id, id)) {
        throw new HttpError('FORBIDDEN', 'এই Booking দেখার অনুমতি নেই');
      }
      return streamBooking(req, res, { hub, user, bookingId: id });
    }
    if (req.method === 'GET' && id && !action) {
      return json(res, 200, { data: hub.snapshot(user.id, id) });
    }
    throw new HttpError('NOT_FOUND', 'Route পাওয়া যায়নি');
  }

// ...
```

server.mjs · JavaScript অংশ 8 / 8 (Part 4 of 4) চলবে: Node.js

```
// ...

  const gateway = attachGateway(server, { hub, users, allowedOrigins,
    heartbeatMs, authTimeoutMs });
  return { server, gateway };
}
```

অজানা Booking ও অন্যের Booking—দুটিতেই একই 403, যাতে কেউ ID অনুমান করে কোন Booking আছে তা জানতে না পারে।

Browser Dashboard

Page-টি ছোট। Token থাকে URL-এর `#` অংশে, যা Browser Server-এ পাঠায় না (শুধু শেখার সুবিধার জন্য; আসল প্রয়োগে Login ও Cookie):

index.html · HTML — চলবে: Browser

```
<!doctype html>
<html lang="bn">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>TravelLive</title>
  <link rel="icon" href="data:,">
  <style>
    body { font-family: system-ui, sans-serif; margin: 2rem; }
    #connection { font-weight: bold; }
    li[data-status="CONFIRMED"] { color: #0a7a3b; }
    li[data-status="CANCELLED"] { color: #b42318; }
  </style>
</head>
<body>
  <h1>TravelLive</h1>
  <p>সংযোগ: <span id="connection" role="status">বিচ্ছিন্ন</span></p>
  <ul id="bookings" aria-live="polite"></ul>
  <!-- Token থাকে URL-এর # অংশে (যেমন #token-rafi:bk-1); তা Server-এ যায় না। -->
  <script type="module" src="/app.mjs"></script>
</body>
</html>
```

`app.mjs` একই `client-state.mjs` ও `backoff.mjs` ব্যবহার করে যা Server-এর Test-এ পরীক্ষিত—Browser ও Node.js দুই জায়গায় একই ES Module। প্রথম অংশ: অবস্থা ও Render:

app.mjs · JavaScript (অংশ ১ / ২ (Part 1 of 2)) — চলবে: Browser

```
import { BookingView } from '/client-state.mjs';
import { reconnectDelay } from '/backoff.mjs';

const [token, ...bookingIds] = location.hash.slice(1).split(/[:,]/);
const view = new BookingView();
const $ = (selector) => document.querySelector(selector);
let attempt = 0;

function render() {
  $('#bookings').replaceChildren(...bookingIds.map((id) => {
    const item = document.createElement('li');
    const booking = view.get(id);
    item.dataset.status = booking?.status ?? 'UNKNOWN';
    item.textContent = `${id}: ${booking?.title ?? ''} — `
```

```
      + `${booking?.status ?? 'অপেক্ষমাণ'} (#${booking?.sequence ?? 0})`;
    return item;
  }));
}

// ...
```

সংযোগ, Auth, Subscribe ও পুনঃসংযোগ। প্রতিবার Auth-এর পর প্রতিটি Booking-এর শেষ দেখা sequence পাঠানো হয়; Server শুধু বাদ পড়াগুলো পাঠায়। ফাঁক ধরা পড়লে Client নিজেই সংযোগ বন্ধ করে, এবং পুনঃসংযোগ তাকে ঠিক করে দেয়:

app.mjs · JavaScript অংশ ২ / ২ (Part 2 of 2) চলবে: Browser

```
// ...

function connect() {
  const socket = new WebSocket(`ws://${location.host}/live`);
  socket.addEventListener('open', () => {
    socket.send(JSON.stringify({ type: 'auth', token }));
  });
  socket.addEventListener('message', ({ data }) => {
    const msg = JSON.parse(data);
    if (msg.type === 'auth.ok') {
      attempt = 0;
      $('#connection').textContent = 'সংযুক্ত';
      // পুনঃসংযোগে শেষ দেখা Sequence পাঠিয়ে শুধু বাদ পড়া Event চাওয়া।
      for (const bookingId of bookingIds) {
        socket.send(JSON.stringify({ type: 'subscribe', bookingId,
          requestId: bookingId, afterSequence: view.lastSequence(bookingId) }));
      }
    }
    if (msg.type === 'snapshot') view.loadSnapshot(msg.snapshot);
    if (msg.type === 'event' && view.apply(msg.event) === 'gap') {
      socket.close(); // ফাঁক: নতুন করে সংযোগ, Server Replay বা Snapshot দেবে
    }
    if (msg.type === 'ack' && !msg.ok) console.error('subscribe', msg.code);
    render();
  });
  socket.addEventListener('close', () => {
    const delay = reconnectDelay(attempt++);
    $('#connection').textContent = `বিচ্ছিন্ন; ${delay} ms পরে আবার চেষ্টা`;
    setTimeout(connect, delay);
  });
}

render();
connect();
```

demo.mjs

পুরো প্রবাহ এক Process-এ, আসল Network সংযোগে: রাফি Subscribe করেন, Agent নিশ্চিত করেন, মিনা অন্যের Booking চাইতে গিয়ে প্রত্যাখ্যাত হন, রাফি Offline থাকাকালে Agent বাতিল করেন, এবং রাফি ফিরে এসে শুধু বাদ পড়া Event পান:

demo.mjs · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Node.js

```
// পুরো প্রবাহ এক Process-এ: Server চালু, দুই Client, HTTP দিয়ে অবস্থা-বদল।
import { createDemo } from './src/demo-data.mjs';
import { createLiveServer } from './src/server.mjs';
import { BookingView } from './src/client-state.mjs';
import { login } from './tests/helpers.mjs';

const { hub, users } = createDemo();
const { server } = createLiveServer({ hub, users, allowedOrigins: [] });
await new Promise((resolve) => server.listen(0, '127.0.0.1', resolve));
const host = `127.0.0.1:${server.address().port}`;
const view = new BookingView();
const ack = (id) => (m) => m.type === 'ack' && m.requestId === id;

// নির্দিষ্ট Sequence পর্যন্ত অপেক্ষা, তারপর নতুন Event-গুলো ক্রমে প্রয়োগ।
async function applyUntil(client, sequence) {
  await client.next((m) => m.event?.sequence === sequence);
  for (const m of client.messages.splice(0)) {
    if (m.type !== 'event') continue;
    const result = view.apply(m.event);
    console.log(`  #${m.event.sequence} ${m.event.type}: ${result}`);
  }
}

async function rafiSubscribes(afterSequence) {
  const rafi = await login(host, 'token-rafi');
  rafi.send({ type: 'subscribe', bookingId: 'bk-1', requestId: 's',
    afterSequence });
  console.log('Rafi subscribe:', (await rafi.next(ack('s'))).mode);
  return rafi;
}

const changeStatus = (token, status) => fetch(
  `http://${host}/api/bookings/bk-1/status`, { method: 'POST',
    headers: { Authorization: `Bearer ${token}` },
    body: JSON.stringify({ status }) }).then((res) => res.status);

// ...
```

demo.mjs · JavaScript অংশ ২ / ২ (Part 2 of 2) চলবে: Node.js

```
// ...

const rafi = await rafiSubscribes(0);
await applyUntil(rafi, 1);
console.log('Agent CONFIRMED:', await changeStatus('token-agent', 'CONFIRMED'));
await applyUntil(rafi, 2);

const mina = await login(host, 'token-mina');
mina.send({ type: 'subscribe', bookingId: 'bk-1', requestId: 'm' });
console.log('Mina subscribe:', (await mina.next(ack('m'))).code);

rafi.socket.close();
await rafi.closed;
console.log('Rafi offline; Agent CANCELLED:',
  await changeStatus('token-agent', 'CANCELLED'));
const again = await rafiSubscribes(view.lastSequence('bk-1'));
await applyUntil(again, 3);
console.log('Rafi-র Screen:', view.get('bk-1'));

for (const client of [mina, again]) client.socket.close();
server.closeAllConnections();
server.close();
```

Output

```
Rafi subscribe: replay
  #1 booking.created: applied
Agent CONFIRMED: 200
  #2 booking.status.changed: applied
Mina subscribe: FORBIDDEN
Rafi offline; Agent CANCELLED: 200
Rafi subscribe: replay
  #3 booking.status.changed: applied
Rafi-র Screen: { title: "Cox's Bazar, ৩ দিন", status: 'CANCELLED', sequence: 3 }
```

দ্বিতীয়বার রাফি `afterSequence: 2` পাঠান, তাই Server শুধু #৩ পাঠায়—#১ ও #২ আবার আসে না। Demo প্রথম সংস্করণে Console-এ Event ছাপত যেই মুহূর্তে আসত; Network-এর সময়ের ওপর নির্ভর করে Line-গুলোর ক্রম কখনো বদলে যেত। এখন `applyUntil` নির্দিষ্ট Event পর্যন্ত অপেক্ষা করে, তারপর জমা Message ক্রমে প্রয়োগ করে—Real-Time Code-এর Test ও Demo-তে এটি একটি দরকারি অভ্যাস।

Test: আসল সংযোগে

Test-গুলো Port 0-তে (Operating System একটি খালি Port দেয়) আসল Server চালায় এবং Node.js-এর অন্তর্নির্মিত WebSocket Client দিয়ে সংযোগ করে। নিচে চারটি Test—অনুমতি বাতিল, অচেনা Origin, সীমা, এবং Heartbeat:

gateway.test.mjs · JavaScript নির্বাচিত অংশ (Selected Excerpt — Complete file in Companion Code) চলবে: Node.js

```
test('অনুমতি বাতিল হলে Server নিজেই Unsubscribe করে', async (t) => {
  const app = await startServer();
  t.after(app.close);
  const rafi = await login(app.host, 'token-rafi');
  rafi.send({ type: 'subscribe', bookingId: 'bk-1', requestId: 'r' });
  await rafi.next(ack('r'));
  app.hub.revokeUser('u-rafi');
  assert.deepEqual(await rafi.next((m) => m.type === 'unsubscribed'),
    { type: 'unsubscribed', bookingId: 'bk-1', reason: 'FORBIDDEN' });
  rafi.socket.close();
});

test('অচেনা Origin-এর Upgrade: 403', async (t) => {
  const app = await startServer();
  t.after(app.close);
  const tryOrigin = (origin) => new Promise((resolve) => {
    const req = http.request(`http://${app.host}/live`, { headers: {
      Connection: 'Upgrade', Upgrade: 'websocket', Origin: origin,
      'Sec-WebSocket-Version': '13',
      'Sec-WebSocket-Key': 'dGhlIHNhbXBsZSBub25jZQ==' } });
    req.on('upgrade', (res, socket) => { socket.destroy(); resolve(101); });
    req.on('response', (res) => resolve(res.statusCode));
    req.end();
  });
  assert.equal(await tryOrigin('https://evil.example'), 403);
  assert.equal(await tryOrigin(ORIGIN), 101);
  assert.equal(app.gateway.metrics.rejectedOrigins, 1);
});

// ...

test('Heartbeat: Pong না দেওয়া Client বিচ্ছিন্ন হয়', async (t) => {
  const app = await startServer({ heartbeatMs: 50 });
  t.after(app.close);
  const zombie = new WebSocket(`ws://${app.host}/live`);
  zombie.pong = () => {}; // মৃত Client-এর ভান: Ping-এর উত্তর দেয় না
  await new Promise((resolve) => zombie.once('open', resolve));
  const code = await new Promise((resolve) => zombie.once('close', resolve));
  assert.equal(code, 1006);
  while (app.gateway.metrics.connections > 0) {
    await new Promise((resolve) => setTimeout(resolve, 10));
  }
});
```

Origin-এর Test-এ Node.js-এর `http.request` দিয়ে হাতে Upgrade অনুরোধ পাঠানো হয়, কারণ Browser বা অন্তর্নির্মিত WebSocket Client নিজের ইচ্ছামতো `Origin` বসাতে দেয় না। Heartbeat-এর Test-এ `ws` Library-র Client-এর `pong` Method খালি করে একটি "মৃত" Client বানানো হয়; Server দুই Heartbeat পরে তাকে বিচ্ছিন্ন করে, এবং Client দেখে 1006 (Close Frame ছাড়া বন্ধ)।

চালানো এবং প্রত্যাশিত আচরণ

Terminal

```
cd travel-live
npm install
npm test
npm run demo
npm start
```

`npm test` ১৮টি Test চালায়: `hub` ৪ (অবস্থা-যন্ত্র, অনুমতি, Replay ও Snapshot, Dispatch-এর সময় অনুমতি), `client-state` ৩ (Duplicate, পুরোনো ও ফাঁক; Backoff; Token Bucket), `http` ৩ (Health, API-র অনুমতি ও ভুল Input, SSE Replay), এবং `gateway` ৮ (আসল WebSocket সংযোগে Auth, অনুমতি, Replay, অনুমতি বাতিল, Origin, সীমা ও Heartbeat):

Output

```
# tests 18
# pass 18
# fail 0
```

`npm start`-এর পরে Browser-এ `http://localhost:3027/#token-rafi:bk-1,bk-2` খুলুন। bk-1 রাফির, তাই দেখা যায়; bk-2 মিনার, তাই Console-এ "subscribe FORBIDDEN" এবং তালিকায় "অপেক্ষমাণ"। এবার README-এর `curl` Command দিয়ে Agent হিসেবে bk-1 নিশ্চিত করুন—Page Refresh ছাড়াই অবস্থা বদলায়। Server বন্ধ করে আবার চালালে Page "বিচ্ছিন্ন; … ms পরে আবার চেষ্টা" দেখায় এবং নিজেই ফিরে আসে (তবে Memory-র অবস্থা Restart-এ মুছে যায়—README-র সীমাবদ্ধতা দেখুন)।

এই বইয়ের QA-তে Dashboard আসল Headless Chrome-এ চালানো হয়েছে: সংযোগ ও Auth, নিজের Booking-এর Replay, অন্যের Booking প্রত্যাখ্যান, Refresh ছাড়া Live Update, Server-এর দিক থেকে জোর করে বিচ্ছিন্ন, Backoff-সহ স্বয়ংক্রিয় পুনঃসংযোগ, এবং বিচ্ছিন্ন থাকাকালে ঘটা বাতিলের Replay—আটটি যাচাইয়ের সবগুলো পাস করেছে। প্রথম চালনায় একটি অপ্রত্যাশিত Console Error ধরা পড়েছিল: Browser `/favicon.ico` চেয়ে 404 পেয়েছিল; `index.html`-এ একটি খালি Icon Link যোগ করে তা দূর করা হয়েছে।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
সংযোগ সঙ্গে সঙ্গে বন্ধ, Code 4401	Auth Message পাঠানো হয়নি বা ভুল Token	`open`-এর পরে প্রথমেই `auth`
Upgrade-এ 403	Page-এর Origin তালিকায় নেই	`allowedOrigins`-এ সঠিক Scheme, Host ও Port
Browser-এ "WebSocket connection failed"	ভুল Path বা Port, অথবা `wss`/`ws` গরমিল	`/live`, আর HTTPS Page-এ `wss://`
Live Update আসে না	Subscribe-এর `ack`-এ `ok: false`	Console-এ `ack`-এর `code` দেখুন
Reconnect-এর পর পুরোনো অবস্থা	`afterSequence` পাঠানো হয়নি বা Snapshot উপেক্ষিত	`lastSequence`, `snapshot` Message
সংযোগ প্রতি কয়েক মিনিটে কাটে	Proxy-র Idle Timeout	Heartbeat-এর বিরতি Timeout-এর চেয়ে কম
Server-এর Memory বাড়তেই থাকে	বন্ধ সংযোগের Subscription মুছছে না	`close`-এ Unsubscribe; `subscriberCount` দেখুন

২৭.১৩ সাধারণ ভুল

ভুল	কেন সমস্যা	সমাধান
সংযুক্ত মানেই অনুমোদিত ধরা	যে কেউ যেকোনো Room চাইতে পারে	প্রতিটি Subscribe-এ Server-এ অনুমতি
শুধু Subscribe-এর সময় অনুমতি যাচাই	Logout বা ভূমিকা বদলের পরেও Event যায়	Dispatch-এর সময়ও যাচাই
`send()` সফল মানেই Client পেয়েছে	Buffer-এ থাকতে পারে, সংযোগ কাটতে পারে	`ack`, sequence ও Replay
পুনঃসংযোগে কিছু হারায় না ভাবা	বিচ্ছিন্ন সময়ের Event হারায়	Last sequence পাঠিয়ে Replay বা Snapshot
eventId ছাড়া Event	Duplicate চেনা যায় না	স্থির Envelope
সঙ্গে সঙ্গে অসীম পুনঃসংযোগ	Server Restart-এর পর Thundering Herd	Backoff + Jitter + সর্বোচ্চ সীমা
সীমাহীন Send Queue	একটি ধীর Client Memory শেষ করে	Bounded Buffer, Coalesce বা Disconnect
`ws`-এর `error` Event-এ Handler নেই	একটি খারাপ Message-এ পুরো Process বন্ধ	প্রতিটি সংযোগে `on('error')`

ভুল	কেন সমস্যা	সমাধান
Token URL-এর Query-তে	Log ও History-তে ফাঁস	প্রথম Message বা Cookie
Redis Pub/Sub-কে টেকসই Queue ভাবা	Offline Subscriber Message হারায়	Database History বা টেকসই Log

২৭.১৪ Debugging নির্দেশিকা

১. **DevTools-এর Network Panel:** WS Filter দিয়ে সংযোগটি খুলুন; "Messages" Tab-এ প্রতিটি পাঠানো ও আসা Frame, আর বন্ধের Close Code দেখা যায়। SSE-র জন্য "EventStream" Tab।

২. **Close Code পড়ুন:** 1006 মানে Network বা Heartbeat; 1009 মানে বড় Message; 4xxx মানে আপনার Application-এর সিদ্ধান্ত।

৩. **Server-এর Metrics:** TravelLive-এ `gateway.metrics` —সক্রিয় সংযোগ, প্রত্যাখ্যাত Origin, ধীর Client ও Protocol Error। সংযোগ শুধু বাড়তে থাকলে কোথাও Close Handler-এ সমস্যা।

৪. **sequence দিয়ে খুঁজুন:** Client যে শেষ sequence দেখিয়েছে আর Server-এর History-র শেষ sequence মেলান; ফাঁক কোথায় তৈরি হলো তা বোঝা যায়।

৫. **Terminal থেকে পরীক্ষা:** Node.js-এর REPL-এ `new WebSocket('ws://localhost:3027/live')` খুলে হাতে Message পাঠান; SSE-র জন্য `curl -N -H "Authorization: Bearer token-rafi" http://localhost:3027/api/bookings/bk-1/stream`।

৬. **সময়-নির্ভর Bug:** Test-এ Heartbeat ও Auth Timeout ছোট করুন (TravelLive-এর `heartbeatMs`, `authTimeoutMs`), এবং অপেক্ষা করুন নির্দিষ্ট Message-এর জন্য, নির্দিষ্ট সময়ের জন্য নয়।

২৭.১৫ পেশাদার অভ্যাস

- **Event-এর একটি লিখিত চুক্তি রাখুন:** Envelope, প্রতিটি type-এর Payload ও version—Client ও Server দুই দল একই Document দেখে।
- **Domain ও Transport আলাদা:** নিয়ম একটি জায়গায় (Hub), WebSocket ও SSE শুধু বাহক।
- **Snapshot সব সময় একটি পথ:** Replay ব্যর্থ হলে Client-এর একটি নিশ্চিত পথ থাকা চাই।
- **At-least-once ধরে নিয়ে Consumer Idempotent।**
- **প্রতিটি সীমা স্পষ্ট:** Message-এর আকার, হার, Subscription-এর সংখ্যা, Buffer, History।
- **Graceful Shutdown:** Deploy-এর সময় সংযোগ বন্ধ করুন (1001), Client Backoff-সহ নতুন Instance-এ যাবে।
- **আসল সংযোগে Test, আর অন্তত একবার আসল Browser-এ:** Mock দিয়ে Close Code, Buffer বা Heartbeat-এর আচরণ ধরা পড়ে না।

২৭.১৬ অনুশীলনী

প্রাথমিক স্তর

১. ৫০০ জন ব্যবহারকারী প্রতি ১০ সেকেন্ডে Polling করেন, আর ঘণ্টায় ২০টি পরিবর্তন হয়। এক ঘণ্টায় কত অনুরোধ, কতগুলো অর্থহীন, আর গড় দেরি কত? একই কাজে SSE-র সুবিধা ও একটি সীমা লিখুন।

২. একটি Function `unsafeKeys(payload)` লিখুন, যা Payload-এর যেকোনো গভীরতায় `password`, `token`, `email`, `phone` বা `cardNumber` Key খুঁজে তার Path ফেরত দেয়।

৩. TravelLive-এ এই Close Code-গুলোর প্রতিটি কখন দেখা যায়: 1000, 1006, 1009, 4401, 4429?

৪. Socket.IO Client কেন একটি সাধারণ WebSocket Server-এ সংযোগ করতে পারে না? আর কখন Socket.IO বেছে নেওয়া যুক্তিসংগত?

মধ্যম স্তর

৫. At-least-once Delivery-র জন্য একটি Idempotent Notification Consumer লিখুন, যা সীমিত সংখ্যক eventId মনে রাখে। এর একটি দুর্বলতা দেখান।

৬. `BookingView`-এর ওপর একটি `receive(view, event, fetchSnapshot)` লিখুন: ফাঁক হলে Snapshot এনে আবার চেষ্টা করে।

৭. `node:test` দিয়ে তিনটি Test লিখুন: অন্য গ্রাহক Subscribe করতে পারেন না; মালিক ও Agent পারেন; অজানা Booking-ও একই Error দেয়।

৮. একটি SSE Frame Parser লিখুন, যা একাধিক `data` Line জোড়া লাগায় এবং `:` দিয়ে শুরু মন্তব্য Line বাদ দেয়।

চ্যালেঞ্জ স্তর

৯. Outbox Relay-তে Retry যোগ করুন: প্রতিটি ব্যর্থতায় `attempts` বাড়বে, তিনবারের পর সারিটি `dead` হবে। Broker প্রথমে বন্ধ, পরে চালু—এমন পরিস্থিতিতে দেখান।

১০. প্রতি সংযোগে সর্বোচ্চ Subscription-এর সংখ্যা সীমিত করুন (যেমন ২০), যাতে একটি Client হাজার Room-এ যোগ দিয়ে Memory শেষ করতে না পারে।

১১. TravelLive-কে তিনটি Instance-এ চালানোর নকশা করুন: Broker, sequence-এর উৎস, Replay-র History, এবং পুনঃসংযোগে অন্য Instance-এ গেলে কী হবে।

১২. TravelLive-এর একটি নিরাপত্তা পর্যালোচনা করুন: Production-এ যাওয়ার আগে অন্তত ছয়টি ফাঁক চিহ্নিত করুন এবং প্রতিটির সমাধান লিখুন।

২৭.১৭ অনুশীলনীর সমাধান

সমাধান ১ — অনুরোধ: ৫০০ × (৩৬০০ ÷ ১০) = ১,৮০,০০০। কাজের উত্তর: ৫০০ × ২০ = ১০,০০০; অর্থহীন ১,৭০,০০০। গড় দেরি বিরতির অর্ধেক, ৫ সেকেন্ড। SSE-তে ঠিক ১০,০০০ Message, দেরি প্রায় শূন্য, এবং Browser নিজেই পুনঃসংযোগ ও `Last-Event-ID` সামলায়। সীমা: একমুখী—Client-এর কিছু পাঠাতে আলাদা HTTP অনুরোধ লাগে; `EventSource` Custom Header পাঠাতে পারে না; আর HTTP/1.1-এ একই Domain-এ সংযোগের সংখ্যা সীমিত।

সমাধান ২

safe-payload.mjs · JavaScript — চলবে: Node.js

```
// Event-এ যা যাবে না: গোপন বা ব্যক্তিগত তথ্য। নিষিদ্ধ Key খোঁজা হয় গভীরেও।
const FORBIDDEN = new Set(['password', 'token', 'email', 'phone', 'cardNumber']);

export function unsafeKeys(value, path = 'payload') {
  if (value === null || typeof value !== 'object') return [];
  return Object.entries(value).flatMap(([key, inner]) => [
    ...(FORBIDDEN.has(key) ? [`${path}.${key}`] : []),
    ...unsafeKeys(inner, `${path}.${key}`),
  ]);
}

console.log(unsafeKeys({ status: 'CONFIRMED' }));
console.log(unsafeKeys({ status: 'CONFIRMED',
  customer: { name: 'Rafi', email: 'rafi@example.com' }, token: 'abc' }));
```

Output

```
[]
[ 'payload.customer.email', 'payload.token' ]
```

Recursion প্রতিটি Object-এর ভেতরে যায়, তাই লুকানো `customer.email`-ও ধরা পড়ে। এই Function Event প্রকাশের আগে একটি Test বা Development-কালীন যাচাই হিসেবে ভালো কাজ করে; তবে আসল সুরক্ষা হলো Payload তৈরিই করা শুধু অনুমোদিত Field থেকে (Allow-list), নিষিদ্ধ তালিকা থেকে নয়।

সমাধান ৩ — 1000: Client বা Server স্বাভাবিকভাবে বন্ধ করলে (যেমন Browser-এর Tab বন্ধ, `socket.close()`)। 1006: Close Frame ছাড়া সংযোগ শেষ—Network কাটা, অথবা Heartbeat-এ Pong না আসায় Server `terminate()` করলে; এটি কেউ পাঠায় না, Client নিজে জানায়। 1009: Message 8 KB-র বেশি (`maxPayload`)। 4401: Auth-এর আগে অন্য Message, ভুল Token, অথবা ৩ সেকেন্ডের মধ্যে Auth না আসা। 4429: Token Bucket খালি—খুব দ্রুত Message।

সমাধান ৪ — Socket.IO নিজের Handshake (প্রথমে একটি HTTP অনুরোধে Session ID), নিজের Packet-এর Format (Event-এর নাম, Acknowledgement-এর ID) এবং প্রয়োজনে Long Polling Transport ব্যবহার করে; সাধারণ WebSocket Server এগুলোর কোনোটিই বোঝে না, তাই দুই পাশে Socket.IO লাগে। এটি যুক্তিসংগত যখন Room, Acknowledgement, স্বয়ংক্রিয় পুনঃসংযোগ ও Multi-Instance Adapter তৈরি অবস্থায় চাই, এবং Client ও Server দুটিই আপনার নিয়ন্ত্রণে। অন্য দলের বা IoT Client-এর সঙ্গে কথা বললে, বা কোনো Standard Protocol লাগলে, সাধারণ WebSocket।

সমাধান ৫

idempotent-consumer.mjs · JavaScript — চলবে: Node.js

```
// At-least-once Delivery: একই Event একাধিকবার আসতে পারে।
// Consumer মনে রাখে কোন eventId-র কাজ হয়ে গেছে (সীমিত Memory-তে)।
function createConsumer(handler, { limit = 3 } = {}) {
  const done = new Set();
  return (event) => {
    if (done.has(event.eventId)) return 'skipped';
    handler(event);
    done.add(event.eventId);
    if (done.size > limit) done.delete(done.values().next().value);
    return 'handled';
  };
}

const sent = [];
const notify = createConsumer((e) => sent.push(`SMS ${e.bookingId}`));
const deliveries = ['e1', 'e2', 'e1', 'e3', 'e2', 'e4', 'e5', 'e1'];
for (const eventId of deliveries) {
  console.log(eventId, notify({ eventId, bookingId: `bk-${eventId}` }));
}
console.log('পাঠানো SMS:', sent.length);
```

Output

```
e1 handled
e2 handled
e1 skipped
e3 handled
e2 skipped
e4 handled
e5 handled
e1 handled
পাঠানো SMS: 6
```

শেষ Line-এ দুর্বলতা দেখা যায়: Memory-তে মাত্র তিনটি ID থাকে, তাই অনেক পরে আসা পুরোনো `e1` আবার SMS পাঠায়। আর Process Restart হলে সব ভুলে যায়। টেকসই সমাধান: Database-এ `processed_events(event_id PRIMARY KEY)`, এবং কাজ ও সেই সারি একই Transaction-এ (অধ্যায় ২৬-এর Idempotency)। যেখানে sequence আছে, সেখানে "শেষ প্রক্রিয়াকৃত sequence" রাখাই যথেষ্ট—তার চেয়ে ছোট সব বাদ।

সমাধান ৬

gap-recovery.mjs · JavaScript — চলবে: Node.js

```
import { BookingView } from '../../project/travel-live/src/client-state.mjs';

// ফাঁক ধরা পড়লে Server-এর Snapshot আনা; Snapshot-এর চেয়ে পুরোনো Event বাদ।
async function receive(view, event, fetchSnapshot) {
  const result = view.apply(event);
  if (result !== 'gap') return result;
  const snapshot = await fetchSnapshot(event.bookingId);
  view.loadSnapshot(snapshot);
  return `gap, snapshot #${snapshot.sequence}, ${view.apply(event)}`;
}

// Server-এর জায়গায় একটি নকল fetch; আসলে GET /api/bookings/:id।
const fakeFetch = async (bookingId) => ({ bookingId, title: 'Sajek',
  status: 'CONFIRMED', sequence: 5 });
const ev = (sequence, status) => ({ eventId: `e${sequence}`,
  bookingId: 'bk-9', sequence, payload: { status } });

const view = new BookingView();
console.log(await receive(view, ev(1, 'PENDING'), fakeFetch));
console.log(await receive(view, ev(6, 'CANCELLED'), fakeFetch));
console.log(await receive(view, ev(4, 'CONFIRMED'), fakeFetch));
console.log(view.get('bk-9'));
```

Output

```
applied
gap, snapshot #5, applied
stale
{ title: 'Sajek', status: 'CANCELLED', sequence: 6 }
```

#৬ এসে ফাঁক দেখায়; Snapshot #৫ পর্যন্ত নিয়ে আসে, তারপর #৬ বৈধভাবে প্রয়োগ হয়। দেরিতে আসা #৪ এখন `stale`—Snapshot তাকে ইতিমধ্যে অন্তর্ভুক্ত করেছে। Snapshot নিজেও যদি Event-এর চেয়ে পুরোনো হয় (একসঙ্গে আরও Event এলে), দ্বিতীয় `apply` আবার `gap` দেবে; আসল Code-এ তখন সংযোগ নতুন করে Replay চাওয়া নিরাপদ।

সমাধান ৭

subscribe.test.mjs · JavaScript চলবে: Node.js

```
import test from 'node:test';
import assert from 'node:assert/strict';
import { LiveHub } from '../../project/travel-live/src/live-hub.mjs';

function hubWithBooking() {
  const hub = new LiveHub();
  hub.addUser({ id: 'rafi', role: 'customer' });
  hub.addUser({ id: 'mina', role: 'customer' });
  hub.addUser({ id: 'nadia', role: 'agent' });
  const { bookingId } = hub.createBooking('rafi', 'Kuakata');
  return { hub, bookingId };
}

test('অন্য গ্রাহক Subscribe করতে পারেন না', () => {
  const { hub, bookingId } = hubWithBooking();
  assert.throws(() => hub.subscribe('mina', bookingId, () => {}),
    { code: 'FORBIDDEN' });
  assert.equal(hub.subscriberCount(bookingId), 0);
});

test('মালিক ও Agent পারেন', () => {
  const { hub, bookingId } = hubWithBooking();
  hub.subscribe('rafi', bookingId, () => {});
  hub.subscribe('nadia', bookingId, () => {});
  assert.equal(hub.subscriberCount(bookingId), 2);
});

test('অজানা Booking-ও FORBIDDEN: অস্তিত্ব ফাঁস হয় না', () => {
  const { hub } = hubWithBooking();
  assert.throws(() => hub.subscribe('rafi', 'bk-999', () => {}),
    { code: 'FORBIDDEN' });
});
```

Output

```
ok 1 - অন্য গ্রাহক Subscribe করতে পারেন না
ok 2 - মালিক ও Agent পারেন
ok 3 - অজানা Booking-ও FORBIDDEN: অস্তিত্ব ফাঁস হয় না
# tests 3
# pass 3
# fail 0
```

তৃতীয় Test নিরাপত্তার একটি সূক্ষ্ম নিয়ম রক্ষা করে: "নেই" ও "অনুমতি নেই"-এর উত্তর আলাদা হলে আক্রমণকারী ID অনুমান করে বুঝতে পারত কোন Booking আছে।

সমাধান ৮

sse-parser.mjs · JavaScript চলবে: Node.js

```
// SSE Frame পড়া: একাধিক data Line জোড়া লাগে, ':' দিয়ে শুরু Line মন্তব্য।
export function parseFrame(frame) {
  const event = { type: 'message', data: [], id: null };
  for (const line of frame.split('\n')) {
    if (line === '' || line.startsWith(':')) continue;
    const colon = line.indexOf(':');
    const field = colon === -1 ? line : line.slice(0, colon);
    const value = colon === -1 ? '' : line.slice(colon + 1).replace(/^ /, '');
    if (field === 'data') event.data.push(value);
    if (field === 'event') event.type = value;
    if (field === 'id') event.id = value;
  }
  return event.data.length ? { ...event, data: event.data.join('\n') } : null;
}

const stream = ': ping\n\n'
  + 'id: 7\nevent: booking.status.changed\ndata: {"a":1,\ndata: "b":2}\n\n'
  + 'data: সাধারণ বার্তা\n\n';
for (const frame of stream.split('\n\n')) {
  const event = parseFrame(frame);
  if (event) console.log(event.id, event.type, event.data.replace('\n', ' '));
  else if (frame) console.log('(মন্তব্য বা ফাঁকা Frame বাদ)');
}
```

Output

```
(মন্তব্য বা ফাঁকা Frame বাদ)
7 booking.status.changed {"a":1, "b":2}
null message সাধারণ বার্তা
```

Specification অনুযায়ী একাধিক `data` Line একটি `\n` দিয়ে জোড়া লাগে, `:`-এর পরের একটি Space বাদ যায়, আর `event` না থাকলে type হয় `message`। আসল Stream-এ একটি Frame কয়েকটি Chunk-এ ভাগ হয়ে আসতে পারে, তাই উদাহরণ ০২-এর মতো Buffer রেখে `\n\n` পর্যন্ত অপেক্ষা করতে হয়।

সমাধান ৯

outbox-retry.mjs · JavaScript চলবে: Node.js

```
import { DatabaseSync } from 'node:sqlite';

const db = new DatabaseSync(':memory:');
db.exec(`CREATE TABLE outbox (id INTEGER PRIMARY KEY, type TEXT NOT NULL,
  attempts INTEGER NOT NULL DEFAULT 0,
```

```
  state TEXT NOT NULL DEFAULT 'pending');
  INSERT INTO outbox (type) VALUES ('booking.confirmed'), ('poison.event');`);

// প্রতিটি ব্যর্থতায় attempts বাড়ে; ৩ বারের পরে 'dead' (মানুষ দেখবেন)।
function relay(publish, maxAttempts = 3) {
  const rows = db.prepare(
    "SELECT * FROM outbox WHERE state = 'pending' ORDER BY id").all();
  for (const row of rows) {
    try {
      publish(row);
      db.prepare("UPDATE outbox SET state = 'sent' WHERE id = ?").run(row.id);
    } catch {
      const attempts = row.attempts + 1;
      const state = attempts >= maxAttempts ? 'dead' : 'pending';
      db.prepare('UPDATE outbox SET attempts = ?, state = ? WHERE id = ?')
        .run(attempts, state, row.id);
    }
  }
}

let brokerUp = false;
const publish = (row) => {
  if (!brokerUp || row.type === 'poison.event') throw new Error('fail');
};
for (const round of [1, 2, 3]) {
  if (round === 2) brokerUp = true; // Broker দ্বিতীয় দফায় ফিরে এল
  relay(publish);
  const rows = db.prepare('SELECT id, attempts, state FROM outbox').all();
  console.log(`দফা ${round}:`,
    rows.map((r) => `${r.id}:${r.state}/${r.attempts}`).join(' '));
}
```

Output

```
দফা 1: 1:pending/1 2:pending/1
দফা 2: 1:sent/1 2:pending/2
দফা 3: 1:sent/1 2:dead/3
```

প্রথম দফায় Broker বন্ধ, দুটিই ব্যর্থ। দ্বিতীয় দফায় সাধারণ Event যায়, কিন্তু "poison" Event—যা সব সময় ব্যর্থ হয়—তৃতীয়বারে `dead`। এটি না থাকলে একটি খারাপ সারি চিরকাল Retry হতো এবং পেছনের সারিগুলো আটকে রাখত। বাস্তবে Retry-এর মাঝে Backoff, `dead` সারির জন্য Alert, এবং কারণ ঠিক করার পর হাতে আবার চালানোর একটি পথ রাখুন।

সমাধান ১০

subscription-limit.mjs · JavaScript — চলবে: Node.js

```
import { LiveHub, LiveError } from '../../project/travel-live/src/live-hub.mjs';

// প্রতি Connection-এ সর্বোচ্চ max-টি Subscription: একটি Client হাজার
// Room-এ যুক্ত হয়ে Server-এর Memory শেষ করতে পারবে না।
function limitedSubscriber(hub, userId, max) {
  const active = new Map();
  return {
    subscribe(bookingId, listener) {
      if (active.size >= max) {
        throw new LiveError('LIMIT', `সর্বোচ্চ ${max}টি Subscription`);
      }
      const result = hub.subscribe(userId, bookingId, listener);
      active.set(bookingId, result.unsubscribe);
      return result;
    },
    unsubscribe(bookingId) {
      active.get(bookingId)?.();
      active.delete(bookingId);
    },
  };
}

const hub = new LiveHub();
hub.addUser({ id: 'nadia', role: 'agent' });
hub.addUser({ id: 'rafi', role: 'customer' });
for (let i = 0; i < 3; i += 1) hub.createBooking('rafi', `Tour ${i + 1}`);

const conn = limitedSubscriber(hub, 'nadia', 2);
for (const id of ['bk-1', 'bk-2', 'bk-3']) {
  try {
    conn.subscribe(id, () => {});
    console.log(id, 'subscribed');
  } catch (error) {
    console.log(id, error.code, error.message);
  }
}
conn.unsubscribe('bk-1');
conn.subscribe('bk-3', () => {});
console.log('bk-3 subscribed after unsubscribe; bk-1 count =',
  hub.subscriberCount('bk-1'));
```

Output

```
bk-1 subscribed
bk-2 subscribed
bk-3 LIMIT সর্বোচ্চ 2টি Subscription
bk-3 subscribed after unsubscribe; bk-1 count = 0
```

TravelLive-এর Gateway-তে এটি সবচেয়ে সহজে যায় `subscribe` Function-এ: `conn.subs.size >= 20` হলে `ack`-এ `ok: false, code: 'LIMIT'`। Unsubscribe করলে জায়গা খালি হয়, তাই বৈধ ব্যবহারকারী আটকে যান না।

সমাধান ১১ — Broker: Redis (Pub/Sub দ্রুত Fan-out-এর জন্য) অথবা টেকসই Log (Redis Streams, NATS JetStream, Kafka)। sequence-এর উৎস: Booking Service, Database Transaction-এর ভেতরে (`UPDATE bookings SET sequence = sequence + 1 ... RETURNING sequence`), Outbox সারির সঙ্গে একসঙ্গে—কোনো Gateway sequence তৈরি করে না। History: Database-এর `booking_events` Table (Outbox-ই হতে পারে), যেখান থেকে যেকোনো Instance Replay দিতে পারে; পুরোনো Event মুছলে Snapshot। প্রতিটি Instance শুধু তার সংযুক্ত Client-দের Room-এর Channel-এ Subscribe করে। পুনঃসংযোগে Client অন্য Instance-এ গেলে কোনো সমস্যা নেই, কারণ Client নিজের শেষ sequence পাঠায় এবং History সব Instance-এর জন্য এক জায়গায়—এই কারণেই অবস্থা Gateway-এর Memory-তে রাখা হয় না।

সমাধান ১২ — (১) স্থির Demo Token: Login, মেয়াদযুক্ত ও বাতিলযোগ্য Session (অধ্যায় ২০)। (২) `ws://` ও `http://`: TLS-যুক্ত Reverse Proxy-র পেছনে `wss://`। (৩) সব অবস্থা এক Process-এর Memory-তে: Database, Outbox ও টেকসই History। (৪) প্রতি IP বা প্রতি ব্যবহারকারী সংযোগের সংখ্যার সীমা নেই: Gateway বা Proxy-তে সীমা। (৫) প্রতি সংযোগে Subscription-এর সীমা নেই: সমাধান ১০। (৬) Audit Log নেই: কে কখন কোন অবস্থা বদলাল, তা টেকসই Log-এ। (৭) Metrics শুধু Memory-তে: Prometheus বা অনুরূপ ব্যবস্থায় প্রকাশ, এবং অস্বাভাবিক হারে Alert (অধ্যায় ২৮)। (৮) Session শেষ হলে খোলা সংযোগ চলতে থাকে: মেয়াদ শেষে `revokeUser`-এর মতো পুনর্যাচাই ও সংযোগ বন্ধ।

২৭.১৮ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. কোনটি দ্বিমুখী, স্থায়ী সংযোগ?

ক) Short Polling

খ) WebSocket

গ) Server-Sent Events

ঘ) `localStorage`

প্রশ্ন ২. SSE-তে Browser পুনঃসংযোগের সময় কোন Header দিয়ে শেষ পাওয়া Event জানায়?

ক) `Authorization`

খ) `If-None-Match`

গ) `Last-Event-ID`

ঘ) `Origin`

প্রশ্ন ৩. একটি Booking-এর Room-এ যোগ দেওয়ার অনুমতি কোথায় যাচাই করা উচিত?

ক) Browser-এর JavaScript-এ

খ) Room-এর নাম গোপন রেখে

গ) শুধু সংযোগের সময় একবার

ঘ) Server-এ, প্রতিটি Subscribe-এ এবং Event পাঠানোর সময়ও

প্রশ্ন ৪. Event-এর `eventId`-এর প্রধান কাজ কী?

ক) Duplicate চেনা

খ) Event-এর ক্রম নির্ধারণ

গ) Payload-এর সংস্করণ জানানো

ঘ) ব্যবহারকারী চেনা

প্রশ্ন ৫. Client-এ আসা Event-এর sequence শেষ দেখা sequence-এর চেয়ে দুই বেশি। সঠিক পদক্ষেপ?

ক) প্রয়োগ করা, কারণ নতুন

খ) বাদ দেওয়া, কারণ Duplicate

গ) প্রয়োগ না করে Replay বা Snapshot চাওয়া

ঘ) Page Reload করা

প্রশ্ন ৬. Redis Pub/Sub সম্পর্কে কোনটি সঠিক?

ক) Offline Subscriber-এর জন্য Message জমা রাখে

খ) শুধু সেই মুহূর্তে সংযুক্ত Subscriber Message পায়

গ) Exactly-once নিশ্চিত করে

ঘ) Message ৩০ দিন রাখে

প্রশ্ন ৭. Transactional Outbox কোন সমস্যার সমাধান করে?

ক) Database-এ লেখা ও Event প্রকাশের মাঝে অসামঞ্জস্য (Dual Write)

খ) ধীর Query

গ) Duplicate Event

ঘ) WebSocket-এর Origin যাচাই

প্রশ্ন ৮. Server Restart-এর পর হাজার Client-এর জন্য সঠিক পুনঃসংযোগ নীতি?

ক) সঙ্গে সঙ্গে, বারবার

খ) কখনো আবার চেষ্টা নয়

গ) প্রতি ১ সেকেন্ডে স্থির বিরতিতে

ঘ) Exponential Backoff, সর্বোচ্চ সীমা ও Jitter

প্রশ্ন ৯. একটি ধীর Client-এর জন্য Server-এর Send Buffer বাড়তেই থাকলে কোনটি সঠিক নয়?

ক) Buffer-এর সীমা ছাড়ালে বিচ্ছিন্ন করা

খ) একই Booking-এর পুরোনো Event একীকরণ

গ) সীমাহীন Queue রেখে অপেক্ষা

ঘ) পুরোনো Event ফেলে Client-কে Snapshot চাওয়ানো

প্রশ্ন ১০. At-least-once Delivery-তে কী ঘটতে পারে, তাই Consumer-কে Idempotent হতে হয়?

ক) একই Event একাধিকবার পৌঁছানো

খ) কোনো Event না পৌঁছানো, এবং তা নিশ্চিত

গ) সব Event-এর বিশ্বব্যাপী ক্রম

ঘ) শূন্য দেরি

২৭.১৯ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–খ · ২–গ · ৩–ঘ · ৪–ক · ৫–গ · ৬–খ · ৭–ক · ৮–ঘ · ৯–গ · ১০–ক

প্রশ্ন	উত্তর	ব্যাখ্যা
১	খ	SSE একমুখী, Polling প্রতিবার নতুন অনুরোধ; WebSocket দুই দিকে একই সংযোগে।
২	গ	TravelLive-এর `sse.mjs` এই Header থেকেই `afterSequence` নেয়।
৩	ঘ	Room-এর নাম অনুমানযোগ্য, আর অনুমতি পরে বাতিল হতে পারে (Hub-এর `#dispatch`)।
৪	ক	ক্রম ও ফাঁকের জন্য `sequence`, সংস্করণের জন্য `version`।
৫	গ	মাঝের Event হারিয়েছে; `BookingView` একে `gap` বলে।
৬	খ	উদাহরণ ০৮-এ Offline Agent #৩ আর পায়নি।
৭	ক	দুটি লেখা একই Transaction-এ; Relay পরে প্রকাশ করে।
৮	ঘ	Jitter Client-দের ছড়িয়ে দেয়; উদাহরণ ০৭-এ ৩ থেকে ৫ সেকেন্ড।
৯	গ	সীমাহীন Queue একটি Client দিয়েই Server-এর Memory শেষ করতে পারে।

প্রশ্ন	উত্তর	ব্যাখ্যা
১০	ক	Retry মানেই Duplicate-এর সম্ভাবনা; উদাহরণ ০৬-এর দ্বিতীয় Relay।

২৭.২০ অধ্যায়ের সারসংক্ষেপ

- Real-Time-এর চার পথ: Polling সরল কিন্তু অপচয়ী; SSE একমুখী HTTP Stream; WebSocket দ্বিমুখী স্থায়ী সংযোগ; Socket.IO তার ওপর Room, Acknowledgement ও পুনঃসংযোগসহ নিজস্ব Protocol।
- WebSocket: HTTP Upgrade, চারটি Lifecycle Event, Close Code; Origin যাচাই, প্রথম Message-এ Auth, আকার ও হারের সীমা।
- স্থির Event Envelope: eventId (Duplicate), sequence (ক্রম ও ফাঁক), version (বিবর্তন); Payload-এ শুধু প্রয়োজনীয় তথ্য।
- Command অনুরোধ, Event ঘটে যাওয়া সত্য; Transactional Outbox Dual Write-এর ফাঁক বন্ধ করে, কিন্তু At-least-once দেয়।
- Exactly-once Effect = At-least-once পৌঁছানো + Idempotent প্রক্রিয়া।
- পুনঃসংযোগে শেষ sequence পাঠিয়ে Replay; History না থাকলে Snapshot।
- Backoff + Jitter, Heartbeat, এবং Bounded Buffer দিয়ে Backpressure।
- একাধিক Instance-এ Pub/Sub দিয়ে Fan-out; কিন্তু Pub/Sub টেকসই নয়, sequence আসে মালিক Service থেকে।
- অনুমতি প্রতিটি Subscribe-এ এবং প্রতিটি Dispatch-এ।

পরের অধ্যায়ে আপনি এই ধরনের Application Docker-এ প্যাক করবেন, CI/CD দিয়ে স্বয়ংক্রিয়ভাবে যাচাই ও Deploy করবেন, এবং Health Check, Log ও Metrics দিয়ে Production-এ পর্যবেক্ষণ করবেন—TravelLive-এর Graceful Shutdown ও Metrics সেখানে কাজে লাগবে।

২৭.২১ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
Polling / Long Polling	বারবার অনুরোধ / খবর না আসা পর্যন্ত আটকে রাখা অনুরোধ।
Server-Sent Events (SSE)	Server থেকে Browser-এ একমুখী HTTP Event Stream।
WebSocket	HTTP Upgrade থেকে তৈরি দ্বিমুখী স্থায়ী সংযোগ।
Close Code	WebSocket বন্ধের কারণের সংখ্যা।
Event Envelope	প্রতিটি Event-এর স্থির কাঠামো।
Command / Event	কাজের অনুরোধ / ঘটে যাওয়া সত্য।

পরিভাষা	অর্থ
Transactional Outbox	Business পরিবর্তন ও প্রকাশিতব্য Event একই Transaction-এ লেখা।
At-least-once / At-most-once	হারায় না কিন্তু Duplicate সম্ভব / Duplicate নেই কিন্তু হারাতে পারে।
Idempotent Consumer	একই Event বারবার পেলেও ফল একবারের মতো।
Replay / Snapshot	বাদ পড়া Event আবার পাঠানো / বর্তমান পূর্ণ অবস্থা।
Exponential Backoff, Jitter	ক্রমশ দ্বিগুণ অপেক্ষা, এলোমেলো বিচ্যুতি।
Heartbeat (Ping/Pong)	সংযোগ জীবিত কি না নিয়মিত যাচাই।
Backpressure	ধীর গ্রাহকের কারণে জমে ওঠা Data সামলানো।
Pub/Sub, Broker	Channel-এ প্রকাশ ও গ্রহণ; মাঝের Message-বাহক।

২৭.২২ আরও পড়ুন

- **MDN — WebSocket API:** `https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API` — Browser-এর WebSocket ও Close Code।
- **MDN — Using server-sent events:** `https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events/Using_server-sent_events` — `EventSource` ও Stream-এর Format।
- **RFC 6455 — The WebSocket Protocol:** `https://www.rfc-editor.org/rfc/rfc6455` — Handshake, Frame ও Close Code।
- **ws — README:** `https://github.com/websockets/ws` — Node.js-এর WebSocket Server, Heartbeat-এর উদাহরণ।
- **Socket.IO — Documentation:** `https://socket.io/docs/v4/` — Room, Acknowledgement ও Redis Adapter।
- **Redis — Pub/Sub:** `https://redis.io/docs/latest/develop/interact/pubsub/` — Delivery-র নিশ্চয়তা ও সীমা।